

Mandatory assignment

Term Structure Models

Benjamin Siddique (vhb925), Leutrim Beluli (wvt136) & Lumi Sogojeva (xmf370)

31 May 2026

1 Foundational models

1. Firstly, notice that

$$dX_t = \sigma_X(t)X_t dW_t^{\mathbb{Q}}$$

Using Itô's formula with $f(t, x) = \log(x)$ we obtain

$$\begin{aligned} df(t, X_t) &= \frac{1}{X_t} dX_t - \frac{1}{2X_t^2} \langle X, X \rangle_t \\ &= \frac{\sigma_X(t)X_t dW_t^{\mathbb{Q}}}{X_t} - \frac{\sigma_X^2(t)X_t^2 dt}{2X_t^2} \\ &= \sigma_X(t) dW_t^{\mathbb{Q}} - \frac{1}{2} \sigma_X^2(t) dt \end{aligned}$$

By definition of an SDE, we have

$$f(t, X_t) = \log(X_t) = \log(x) + \int_0^t \sigma_X(s) dW_s^{\mathbb{Q}} - \frac{1}{2} \int_0^t \sigma_X^2(s) ds$$

and hence

$$X_t = x \exp \left\{ \int_0^t \sigma_X(s) dW_s^{\mathbb{Q}} - \frac{1}{2} \int_0^t \sigma_X^2(s) ds \right\}$$

The solution to Y_t is much simpler, since by definition of the dynamics of Y_t ,

$$Y_t = Y_0 + \int_0^t \sigma_Y(s) dW_s^{\mathbb{Q}}$$

For the distribution of Y_t we use the fact that for a deterministic $\sigma(t)$ of time

$$\int_0^t \sigma(s) dW_s^{\mathbb{Q}} \sim N \left(0, \int_0^t \sigma^2(s) ds \right)$$

Using this, it is easy to see that $Y_t \sim N \left(Y_0, \int_0^t \sigma_Y^2(s) ds \right)$. By the same argument

$$\log(X_t) \sim N \left(\log(x) - \frac{1}{2} \int_0^t \sigma_X^2(s) ds, \int_0^t \sigma_X^2(s) ds \right)$$

and therefore X_t is LogNormal.

Y_t is a martingale since for $0 \leq u \leq t$,

$$\begin{aligned} E[Y_t | \mathcal{F}_u] &= E \left[Y_0 + \int_0^u \sigma_Y(s) dW_s^{\mathbb{Q}} + \int_u^t \sigma_Y(s) dW_s^{\mathbb{Q}} | \mathcal{F}_u \right] \\ &= E \left[Y_u + \int_u^t \sigma_Y(s) dW_s^{\mathbb{Q}} | \mathcal{F}_u \right] \\ &= Y_u \end{aligned}$$

X_t is also a martingale. Of course, $J_t := \int_0^t \sigma_X(s) dW_s^{\mathbb{Q}}$ is a driftless Itô process and such it is a continuous local martingale, which implies the stochastic exponential $\mathcal{E}(J)_t$ is a continuous local martingale by Lemma 4.2. But notice,

$$X_t = x \mathcal{E}(J)_t$$

and therefore X_t is a continuous local martingale. Since $\sigma_X^2(\cdot) \in \mathcal{L}^2$, Novikov's condition is immediately satisfied and therefore X_t is a true martingale (Theorem 5.23 in [?]).

2. Simple R code gives the following density plot for X_t and Y_t at $t = 5$

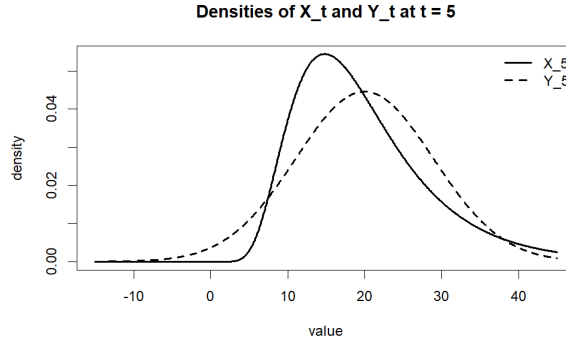


Figure 1: Densities for X_t and Y_t for $t = 5$

The density of Y_5 shows a symmetric and normal-shaped curve unlike X_5 that is asymmetric and clearly right skewed, since it is lognormal. Also note that this is the reason for X_5 having 0 density for negative values.

3. Define $v_X^2 = \int_0^T \sigma_X^2(s)ds$ and $v_Y^2 = \int_0^T \sigma_Y^2(s)ds$. For X_T , notice that the mean can be rewritten as

$$E^{\mathbb{Q}}[(X_T - K)^+] = E^{\mathbb{Q}}[X_T 1(X_T > K)] - K \mathbb{Q}(X_T > K)$$

Let's start with the second term. From 1., $J_t \sim N(0, v_X^2)$ and therefore $J_t \stackrel{d}{=} v_X Z$ for $Z \sim N(0, 1)$. This is useful since

$$\begin{aligned} \mathbb{Q}(X_T > K) &= \mathbb{Q}\left(x \exp\left\{\int_0^T \sigma_X(s) dW_s^{\mathbb{Q}} - \frac{1}{2} \int_0^T \sigma_X^2(s) ds\right\} > K\right) \\ &= \mathbb{Q}\left(x \exp\left\{v_X Z - \frac{1}{2} v_X^2\right\} > K\right) \\ &= \mathbb{Q}\left(Z > \frac{\log(K/x) + 1/2 v_X^2}{v_X}\right) \end{aligned}$$

Defining

$$d_2 = \frac{\log(x/K) - 1/2 v_X^2}{v_X}$$

we get $\mathbb{Q}(Z > -d_2) = 1 - \mathbb{Q}(Z < -d_2) = 1 - \Phi(-d_2) = \Phi(d_2)$.

For the first term, we calculate

$$\begin{aligned} E^{\mathbb{Q}}[X_T 1(X_T > K)] &= x \exp\left\{-\frac{1}{2} v_X^2\right\} E[\exp\{v_X Z\} 1(Z > -d_2)] \\ &= x \exp\left\{-\frac{1}{2} v_X^2\right\} \int_{-d_2}^{\infty} \exp\{v_X z\} \varphi(z) dz \\ &= x \exp\left\{-\frac{1}{2} v_X^2\right\} \exp\left\{\frac{1}{2} v_X^2\right\} \int_{-d_2}^{\infty} \varphi(z - v_X) dz \\ &= x \int_{-d_2 - v_X}^{\infty} \varphi(u) du \\ &= x \Phi(d_2 + v_X) \end{aligned}$$

Letting

$$d_1 = d_2 + v_X = \frac{\log(x/K) + 1/2 v_X^2}{v_X}$$

we obtain

$$E^{\mathbb{Q}}[(X_T - K)^+] = x\Phi(d_1) - K\Phi(d_2)$$

For Y_T , similarly to before, we can write $Y_T = x + v_Y Z$ for $Z \sim N(0, 1)$. With the same argument as before we can split the mean

$$\begin{aligned} E^{\mathbb{Q}}[(Y_T - K)^+] &= E^{\mathbb{Q}}[(x + v_Y Z - K)^+] \\ &= E^{\mathbb{Q}}[(x + v_Y Z)1(x + v_Y Z > K)] - K\mathbb{Q}(x + v_Y Z > K) \\ &= (x - K)\mathbb{Q}(x + v_Y Z > K) + v_Y E^{\mathbb{Q}}[Z1(x + v_Y Z > K)] \end{aligned}$$

Let $d_3 = \frac{x-K}{v_Y}$. Then the event $x + v_Y Z > K$ is instead $Z > -d_3$ and we obtain

$$E^{\mathbb{Q}}[(Y_T - K)^+] = (x - K)\mathbb{Q}(Z > -d_3) + v_Y E^{\mathbb{Q}}[Z1(Z > -d_3)]$$

The first term, by the same arguments as before, is just

$$(x - K)\mathbb{Q}(Z > -d_3) = (x - K)\Phi(d_3)$$

For the first term, we note that $\varphi'(x) = -x\varphi(x)$. With this in mind,

$$\begin{aligned} E^{\mathbb{Q}}[Z1(Z > -d_3)] &= \int_{-d_3}^{\infty} z\varphi(z)dz \\ &= - \int_{-d_3}^{\infty} \varphi'(z)dz \\ &= \varphi(-d_3) \\ &= \varphi(d_3) \end{aligned}$$

and therefore

$$E^{\mathbb{Q}}[(Y_T - K)^+] = (x - K)\Phi(d_3) + v_Y\varphi(d_3)$$

4. Ignoring the drift, neither model is universally better for modelling the short rate r_t . Specifically, Y_t is better suited if negative short rates are allowed, as in the Gaussian short-rate models like the Vasicek-Hull-White. X_t is a more appropriate choice for the short rate, if we want to force the short rates to stay strictly positive, since X_t is lognormal and therefore has strictly positive support. This is closer to that of the Dothan lognormal short rate model. However, strict positivity might be too restrictive in a modern view.

For the ZCB $P(t, T)$, X_t is a better suited model, since $X_t > 0$ a.s. However the bond prices are often below 1, and a lognormal model does not guarantee this. Hence, X_t is not satisfactory but still better than Y_t .

The Libor rate $L(t, T, T + \Delta)$ rates are positive, so X_t is again a more appropriate choice. Y_t can become negative and therefore a worse choice for the Libor rate.

The Libor spread $L(t, T, T + \Delta_1) - L(t, T, T + \Delta_2)$ can become both positive and negative. So X_t is ruled out since it is again always positive. Y_t is here a more natural choice, since it can take on any value.

The stock price is always positive, and therefore X_t is the only appropriate choice. It should be noted that the stock price could hit 0, which X_t cannot model. In those cases one would probably need to add a default component to more accurately model the fluctuations of the stock price.

2 A stochastic volatility term structure model

$$dr(t) = \kappa_r(\theta_r - r(t))dt + \sigma_r\sqrt{\alpha + \beta v(t)}dW_1^{\mathbb{Q}}(t)$$

$$dv(t) = \kappa_v(\theta_v - v(t))dt + \sigma_v\sqrt{\alpha + \beta v(t)}dW_1^{\mathbb{Q}}(t)$$

$$d\langle W_1^{\mathbb{Q}}, W_2^{\mathbb{Q}} \rangle = \rho dt$$

$$\mathbf{X}_t = (r(t), v(t))^{\top}$$

1. It should be trivial to see:

$$\mu(X_t) = \begin{pmatrix} \kappa_r(\theta_r - r(t)) \\ \kappa_v(\theta_v - v(t)) \end{pmatrix} = \begin{pmatrix} \kappa_r\theta_r \\ \kappa_v\theta_v \end{pmatrix} + \begin{pmatrix} -\kappa_r & 0 \\ 0 & -\kappa_v \end{pmatrix} \begin{pmatrix} r(t) \\ v(t) \end{pmatrix}$$

$$\sigma(X_t) = \begin{pmatrix} \sigma_r\sqrt{\alpha + \beta v(t)} & 0 \\ 0 & \sigma_v\sqrt{\alpha + \beta v(t)} \end{pmatrix}$$

For the drift we simply identify K_0 and K_1 using $\mu(x) = K_0 + K_1x$, which gives us:

$$K_0 = \begin{pmatrix} \kappa_r\theta_r \\ \kappa_v\theta_v \end{pmatrix}, \quad K_1 = \begin{pmatrix} -\kappa_r & 0 \\ 0 & -\kappa_v \end{pmatrix}$$

Since the Wiener process paths are correlated, we apply a 2d Cholesky decomposition before determining the instantaneous variance. This would leave us with:

$$\begin{aligned} \tilde{\sigma}(X_t) &= \begin{pmatrix} \sigma_r\sqrt{\alpha + \beta v(t)} & 0 \\ 0 & \sigma_v\sqrt{\alpha + \beta v(t)} \end{pmatrix} \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix} \\ &= \begin{pmatrix} \sigma_r\sqrt{\alpha + \beta v(t)} & 0 \\ \rho\sigma_v\sqrt{\alpha + \beta v(t)} & \sigma_v\sqrt{(\alpha + \beta v(t))(1 - \rho^2)} \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \tilde{\sigma}(X_t)\tilde{\sigma}(X_t)^{\top} &= \begin{pmatrix} \sigma_r^2(\alpha + \beta v(t)) & \rho\sigma_r\sigma_v(\alpha + \beta v(t)) \\ \rho\sigma_r\sigma_v(\alpha + \beta v(t)) & \rho^2\sigma_v^2(\alpha + \beta v(t)) + \sigma_v^2(\alpha + \beta v(t))(1 - \rho^2) \end{pmatrix} \\ &= \begin{pmatrix} \sigma_r^2(\alpha + \beta v(t)) & \rho\sigma_r\sigma_v(\alpha + \beta v(t)) \\ \rho\sigma_r\sigma_v(\alpha + \beta v(t)) & \sigma_v^2(\alpha + \beta v(t)) \end{pmatrix} \\ &= \alpha \begin{pmatrix} \sigma_r^2 & \rho\sigma_r\sigma_v \\ \rho\sigma_r\sigma_v & \sigma_v^2 \end{pmatrix} + \beta v(t) \begin{pmatrix} \sigma_r^2 & \rho\sigma_r\sigma_v \\ \rho\sigma_r\sigma_v & \sigma_v^2 \end{pmatrix} \end{aligned}$$

Once again, all that is left for us to do is to identify the affine H -parameters using the equation $(\tilde{\sigma}(x)\tilde{\sigma}(x)^{\top})_{ij} = H_{0,ij} + H_{1,ij} \cdot x$:

$$H_0 = \alpha \begin{pmatrix} \sigma_r^2 & \rho\sigma_r\sigma_v \\ \rho\sigma_r\sigma_v & \sigma_v^2 \end{pmatrix}, \quad H_1 = \beta \begin{pmatrix} \begin{pmatrix} 0 \\ \sigma_r^2 \end{pmatrix} & \begin{pmatrix} 0 \\ \rho\sigma_r\sigma_v \end{pmatrix} \\ \begin{pmatrix} 0 \\ \rho\sigma_r\sigma_v \end{pmatrix} & \begin{pmatrix} 0 \\ \sigma_v^2 \end{pmatrix} \end{pmatrix}$$

2. Applying Feynman-Kac on the Zero-Coupon-Bond we get:

$$\frac{\partial P}{\partial t} + \mu_r \frac{\partial P}{\partial r} + \frac{1}{2}(\sigma\sigma^{\top})_{rr} \frac{\partial^2 P}{\partial r^2} + \mu_v \frac{\partial P}{\partial v} + \frac{1}{2}(\sigma\sigma^{\top})_{vv} \frac{\partial^2 P}{\partial v^2} + (\sigma\sigma^{\top})_{vr} \frac{\partial^2 P}{\partial v \partial r} = rP$$

Now we can enforce the separation of variables ansatz $P(t, T) = e^{\alpha_z(T-t) + \beta_z(T-t) \cdot X_t}$. In order to do this cleanly we define $\tau = T - t$, which also implies $\frac{\partial P}{\partial t} = \frac{\partial P}{\partial \tau} \frac{\partial \tau}{\partial t} = -\frac{\partial P}{\partial \tau}$. We are left with:

$$-\frac{\partial P}{\partial \tau} + \mu_r \frac{\partial P}{\partial r} + \frac{1}{2}(\sigma\sigma^{\top})_{rr} \frac{\partial^2 P}{\partial r^2} + \mu_v \frac{\partial P}{\partial v} + \frac{1}{2}(\sigma\sigma^{\top})_{vv} \frac{\partial^2 P}{\partial v^2} + (\sigma\sigma^{\top})_{vr} \frac{\partial^2 P}{\partial v \partial r} = rP$$

$$\begin{aligned}
& - (\alpha'_z(\tau) + \beta'_{z,r}(\tau)r + \beta'_{z,v}(\tau)v) P + \kappa_r(\theta_r - r)P\beta_{z,r}(\tau) + \frac{1}{2}\sigma_r^2(\alpha + \beta v)P\beta_{z,r}(\tau)^2 \\
& + \kappa_v(\theta_v - v)P\beta_{z,v}(\tau) + \frac{1}{2}\sigma_v^2(\alpha + \beta v)P\beta_{z,v}(\tau)^2 + \rho\sigma_r\sigma_v(\alpha + \beta v)\beta_{z,r}(\tau)\beta_{z,v}(\tau)P - rP = 0
\end{aligned}$$

Dividing through by P and collecting r , v and constant terms separately gives us:

$$\begin{aligned}
& - (\beta'_{z,r}(\tau) + \kappa_r\beta_{z,r} + 1) r \\
& + \left(-\beta'_{z,v}(\tau) + \frac{1}{2}\sigma_r^2\beta_{z,r}(\tau)^2 - \kappa_v\beta_{z,v}(\tau) + \frac{1}{2}\sigma_v^2\beta_{z,v}(\tau)^2 + \rho\sigma_r\sigma_v\beta_{z,r}(\tau)\beta_{z,v}(\tau) \right) v \\
& - \alpha'_z(\tau) + \kappa_r\theta_r\beta_{z,r}(\tau) + \kappa_v\theta_v\beta_{z,v}(\tau) + \frac{1}{2}\sigma_r^2\alpha\beta_{z,r}(\tau)^2 + \frac{1}{2}\sigma_v^2\alpha\beta_{z,v}(\tau)^2 + \rho\sigma_r\sigma_v\alpha\beta_{z,r}(\tau)\beta_{z,v}(\tau) = 0
\end{aligned}$$

For this relation to hold for all τ , we must have the coefficients for r and v as well as the final constant term be zero, which leaves us with the following system of Ricatti differential equations:

$$\beta'_{z,r}(\tau) = -\kappa_r\beta_{z,r} - 1$$

$$\beta'_{z,v}(\tau) = \frac{1}{2}\beta_{z,r}(\tau)^2 + \sigma_v^2\beta_{z,v}(\tau)^2 + 2\rho\sigma_r\sigma_v\beta_{z,r}(\tau)\beta_{z,v}(\tau) - \kappa_v\beta_{z,v}(\tau)$$

$$\alpha'_z(\tau) = \frac{1}{2}\alpha_{z,r}(\tau)^2 + \sigma_v^2\alpha_{z,v}(\tau)^2 + 2\rho\sigma_r\sigma_v\alpha_{z,r}(\tau)\alpha_{z,v}(\tau) + \kappa_r\theta_r\beta_{z,r}(\tau) + \kappa_v\theta_v\beta_{z,v}(\tau)$$

We could also write it up in terms of its affine characteristics:

$$\beta'_z(\tau) = K_1^\top \beta_z(\tau) + \frac{1}{2}\beta_z(\tau)^\top H_1 \beta_z(\tau) - \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$\alpha'_z(\tau) = K_0^\top \beta_z(\tau) + \frac{1}{2}\beta_z(\tau)^\top H_0 \beta_z(\tau)$$

Solving these differential equations using the RK45-scheme we can plot the α - and β -curves along with the ZCB-price $\{P(t, t + \Delta)\}_{0 \leq \Delta \leq 30}$ and yield-curve $\{y(t, t + \Delta)\}_{0 \leq \Delta \leq 30}$. Note that, due to the separation of variables ansatz, the condition $P(T, T) = 1$ implies $\alpha(0) = 0$ and $\beta(0) = 0$.

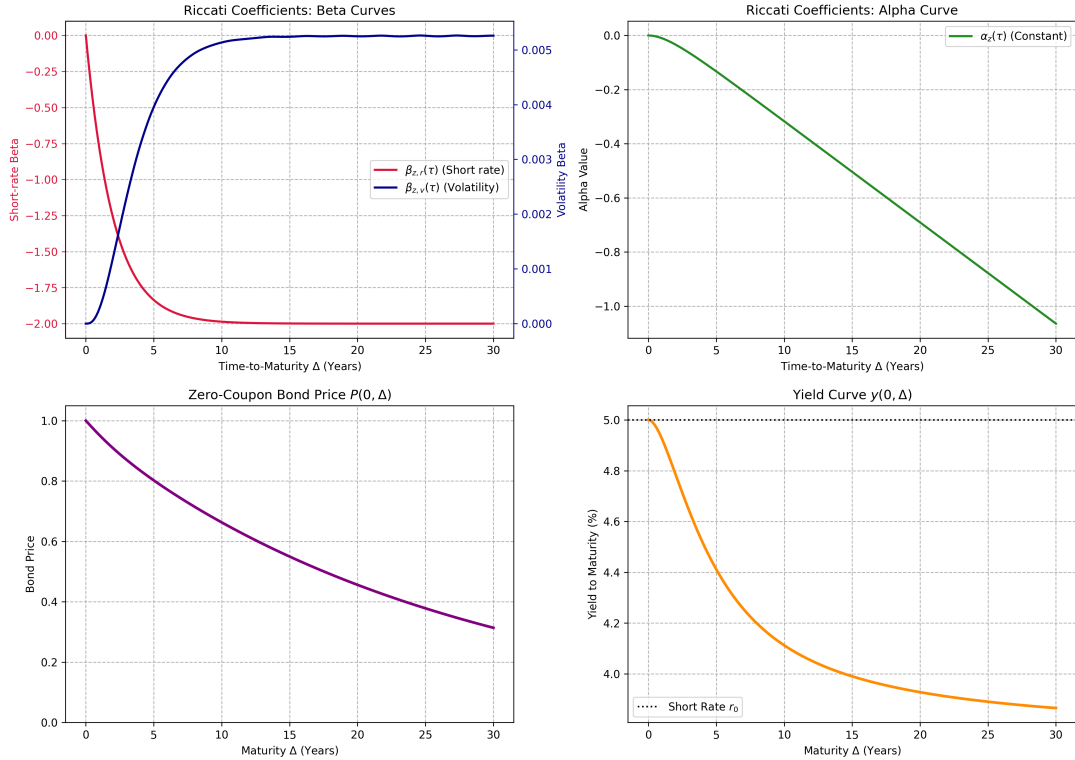


Figure 2: Curves computed using the RK45-scheme with a timestep of 0.1 years.

3. We use the Euler-Maruyama scheme to simulate the paths. The pseudocode for our simulation is given below.

Algorithm 1 General simulation layout

```

for  $i$  in range  $N_{simulations}$  do
  for  $j$  in range  $N_{steps}$  do
     $Z_1, Z_2$  = two randomly generated, normally distributed variables.
     $W_r = Z_1$ 
     $W_v = \rho Z_1 + \sqrt{1 - \rho^2} Z_2$ 
     $r_{next} = r_{now} + \kappa_r(\theta_r - r_{now})dt + \sigma_r \sqrt{(\alpha + \beta v)dt} W_r$ 
     $v_{next} = v_{now} + \kappa_v(\theta_v - v_{now})dt + \sigma_v \sqrt{(\alpha + \beta v)dt} W_v$ 
     $D_{next} = D_{now} - r_{now}dt$ 
     $r_{now} = r_{next}$ 
     $v_{now} = v_{next}$ 
     $D_{now} = D_{next}$ 
     $t = t + dt$ 
    if  $t > T_{maturity}$  then
       $P = \exp(D_{next})$ 
       $P_{list}[i] = P$ 
      break loop
    end if
  end for
end for
 $P_{mean} = \text{mean}(P_{list})$ 

```

The algorithm is once again ran in python, where we have made use of numpy-vectorization for faster computation. We also leave out the if-statement in the end and instead define a maximum maturity time T_{max} , which determines when our simulation is terminated. This way we are able to log the ZCB-price for different maturities along the way. In the below figure we have averaged out simulation results from 10k paths at different maturity dates:

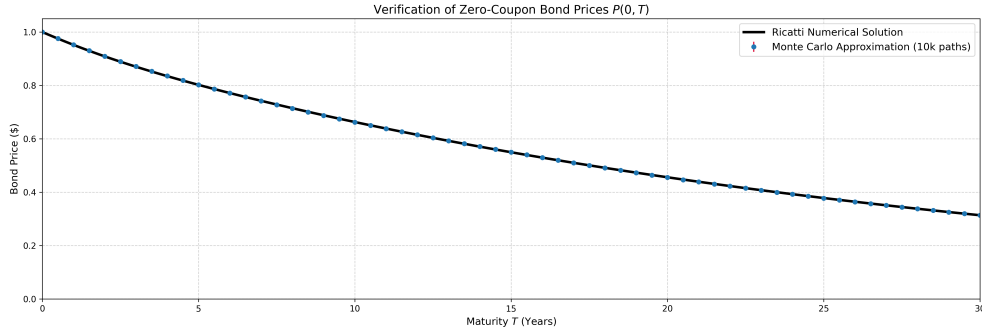


Figure 3: Zero-Coupon Bond prices for different maturities. The plot is identical to the one in 2, but overlaid with points that indicate the mean price obtained from simulating 10k interest rate- and volatility paths. For this simulation we used $N_{steps} = 1500$ and $T_{max} = 30$ years, resulting in timesteps of size $dt = 0.02$ years.

The simulation results are highly consistent with our numerical solution to the system of Ricatti ODEs obtained in the previous section. The errors on the mean values are barely visible on the plot and are generally of the order $\sim 10^{-3}$, further confirming the fact our earlier derived results are indeed accurate.

4. Plotting 20 sample interest rate paths is pretty straightforward from reusing the above code.

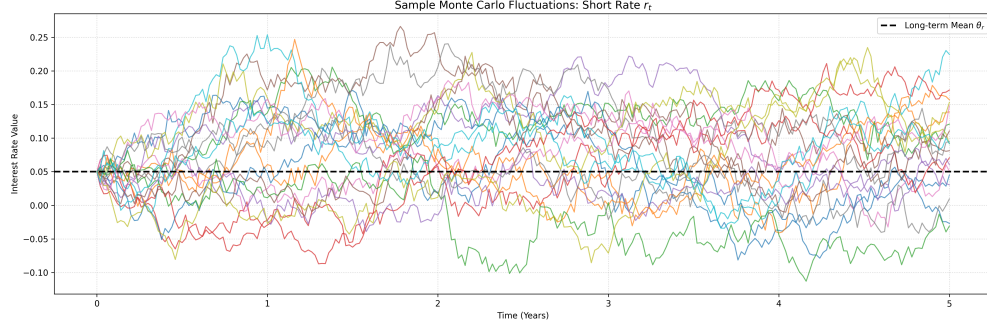


Figure 4: Sample of 20 interest-rate paths using the same time resolution as in the previous section.

As clearly seen on the figure, negative interest rates are possible with this model, and given that they appear for multiple paths in this small sample, they are far from improbable. Plotting the CDF shows this clearly:

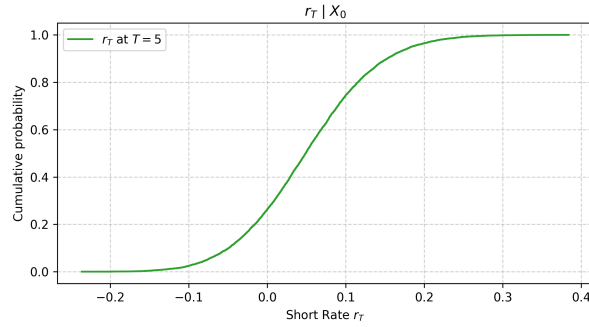


Figure 5: Cumulative distribution of final interest rate values after 5 years.

From the above figure we can clearly see that $\sim 1/4$ of interest rate values are negative at time $T = 5$, which is roughly consistent with what we saw in the sample path fluctuations. Next we determine the CDFs of the zero-coupon bond prices for 1 and 10 years in the future, starting from $T = 5$. This is determined empirically, by tracking how much the interest rate (i.e. discount factor) paths change during that interval.

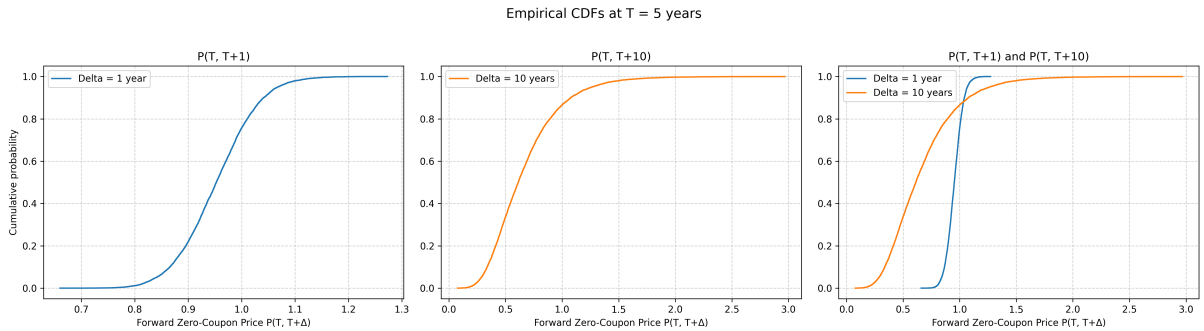


Figure 6: Cumulative distribution functions for $P(T, T + 1)$ and $P(T, T + 10)$ with $T = 5$.

Clearly the Zero-Coupon Bond with the larger time interval has a bigger variation in prices, which makes sense, because the stochasticity of the interest rate paths has had more time to influence the system. The Zero-Coupon Bond with a one-year maturity is also more symmetric and has higher prices than its counterpart, suggesting that it is a more stable investment.

5. We consider the transform

$$\psi(y, X_t, t, \Delta) = E^{\mathbb{Q}}[\mathbf{1}_{\{\beta_z(\Delta) \cdot X_T \leq \log(y) - \alpha_z(\Delta)\}}],$$

which is just an expression for the risk-neutral CDF of the ZCB-price $P(T, T + \Delta)$. We can easily see this by just writing it up in terms of our initial ansatz:

$$P(T, T + \Delta) = e^{\alpha_z(\Delta) + \beta_z(\Delta) \cdot X_t}$$

$$\log(P(T, T + \Delta)) = \alpha_z(\Delta) + \beta_z(\Delta) \cdot X_t$$

$$\log(P(T, T + \Delta)) - \alpha_z(\Delta) = \beta_z(\Delta) \cdot X_t$$

Now we may redefine the indicator condition as:

$$\log(P(T, T + \Delta)) - \alpha_z(\Delta) \leq \log(y) - \alpha_z(\Delta)$$

$$P(T, T + \Delta) \leq y$$

Thus ψ is just the CDF for $P(T, T + \Delta)$, with y determining the upper limit of the of the price. We can determine it by finding the conditional characteristic function of the random variable $Z = \beta(\Delta)X_T$:

$$\phi_Z(k) = E[e^{ikZ}|X_t] = E[e^{ik\beta(\Delta)X_T}|X_t]$$

Due to X_t being a time-homogenous affine process we may, as stated in the first chapter of Duffie (2008), write the above equation as:

$$\phi_Z(k) = e^{A(\tau, k) + B(\tau, k)X_t}$$

where $A(\tau, k)$ and $B(\tau, k)$ are solutions to the Ricatti ODEs presented earlier (with the exception of the -1 term in the $\beta'_{z,r}$ ODE, due to no discounting in this case), with the boundary conditions $A(0, k) = 0$ and $B(0, k) = ik\beta(\Delta)$. We can then determine ψ , using the Gil-Pelaez theorem:

$$\begin{aligned} \psi(y, X_t, t, \Delta) &= \mathbb{P}(P(T, T + \Delta) \leq y) = \mathbb{P}(Z \leq \log(y) - \alpha_z(\Delta)) \\ &= \frac{1}{2} - \frac{1}{\pi} \int_0^\infty \frac{1}{k} \text{Im} \left[\phi_Z(k) e^{-ik(\log(y) - \alpha_z(\Delta))} \right] dk \end{aligned}$$

The characteristic function is determined by solving the Ricatti equations numerically, and the whole integral is computed using the trapezoidal rule. We compare our analytical estimates against the empirical cdfs derived in the previous section, which yields the following graphs:

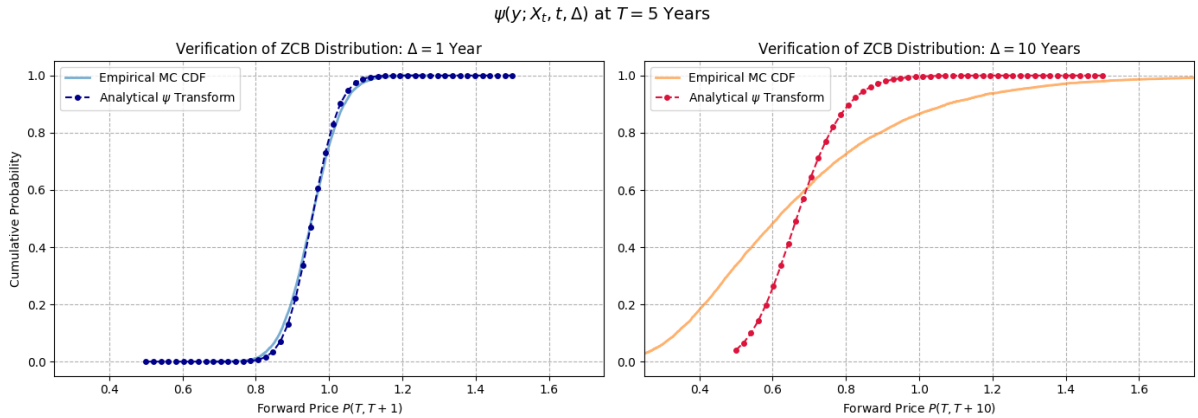


Figure 7: 50 values of ψ calculated for $\Delta = 1$ and $\Delta = 10$ respectively, with $y \in [0.5, 1.5]$.

6. Notice, since $\rho_0 = 0$ and $\rho_1 \cdot \mathbf{X}_t = r(t)$, we have

$$\begin{aligned} G_1(x) &= G_{0,\beta_z(\Delta)}[\log(x) - \alpha_z(\Delta)] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r(u)du} \mathbf{1}_{\{\beta_z(\Delta) \cdot \mathbf{X}_T \leq \log(x) - \alpha_z(\Delta)\}} \right] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r(u)du} \mathbf{1}_{\{P(T, T+\Delta) \leq x\}} \right] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r(u)du} \cdot 1 \cdot \mathbf{1}_{\{P(T, T+\Delta) \leq x\}} \right], \end{aligned}$$

where we again have used our initial ansatz for the indicator condition. So G_1 is just the time- t discounted value of receiving 1 at time T only in states where the future bond price $P(T, T + \Delta)$ is less than x .

For G_2 , we calculate again using the initial ansatz for the indicator condition

$$\begin{aligned} G_2(x) &= e^{\alpha_z(\Delta)} G_{\beta_z(\Delta), \beta_z(\Delta)}[\log(x) - \alpha_z(\Delta)] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r(u)du} \cdot e^{\alpha_z(\Delta) + \beta_z(\Delta) X_T} \cdot \mathbf{1}_{\{P(T, T+\Delta) \leq x\}} \right] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r(u)du} \cdot P(T, T + \Delta) \cdot \mathbf{1}_{\{P(T, T+\Delta) \leq x\}} \right] \end{aligned}$$

Therefore G_2 is just the time- t discounted value of receiving the future bond price $P(T, T + \Delta)$ again under the condition that the bond price is less than x .

Now assume $t = 0$. With $T = 5$, $\Delta = 1$, we have:

$$G_1(x) = E_0^{\mathbb{Q}} \left[e^{-\int_0^T r(u)du} \cdot 1 \cdot \mathbf{1}_{\{P(5,6) \leq x\}} \right]$$

and

$$G_2(x) = E_0^{\mathbb{Q}} \left[e^{-\int_0^T r(u)du} \cdot P(5, 6) \cdot \mathbf{1}_{\{P(5,6) \leq x\}} \right]$$

Of course, if $x = P(t, T + \Delta)/P(t, T)$ then, with $T = 5$ and $\Delta = 1$, we have

$$x = P(0, 6)/P(0, 5) = e^{\alpha_z(6) - \alpha_z(5) + (\beta_z(6) - \beta_z(5))X_0}$$

Also note from before that

$$P(5, 6) \leq x \Leftrightarrow \beta_z(1) \cdot X_5 \leq y$$

where $y = \log(x) - \alpha_z(1)$. The α - and β -functions can be estimated numerically, as described in 2.2. Therefore, we only need the G -transform from Proposition 2 in DPS to compute the values of G_1 and G_2 . Note that, the affine discounted transform is

$$\phi(k) = E_0^{\mathbb{Q}} \left[e^{-\int_0^5 r(u)du} e^{k \cdot X_5} \right]$$

Since X_t is affine, we have

$$\phi(k) = e^{A(5,k) + B(5,k) \cdot X_0}$$

where $A(5, k)$ and $B(5, k)$ are once again solutions to the Ricatti ODEs presented in 2.2, with boundary conditions $A(0, k) = 0$ and $B(0, k) = k$. Thus, by DPS Proposition 2 we can then compute G_1 and G_2 by

$$G_1(x) = \frac{\phi(0)}{2} - \frac{1}{\pi} \int_0^\infty \text{Im} \left[\frac{\phi(iv\beta_z(1))e^{-ivy}}{v} \right] dv$$

and

$$G_2(x) = e^{\alpha_z(1)} \left(\frac{\phi(\beta_z(1))}{2} - \frac{1}{\pi} \int_0^\infty \text{Im} \left[\frac{\phi((1+iv)\beta_z(1))e^{-ivy}}{v} \right] dv \right)$$

In addition to this, we verify the values of G_1 and G_2 at $t = 0$ using Monte Carlo methods, by tracking the discount paths. Say we want G_1 at $\Delta = 1$ and $T = 5$, then we simply determine $P(5, 6) = e^{\alpha_z(1) + \beta(1) \cdot X_5}$ for all paths. We then assign each path a value of 1 or 0, based on if they fulfill the indicator condition $\{P(5, 6) \leq x\}$. Then we simply multiply with the empirical discount factor, which is based on the stochastic interest rate: $df(5) = e^{-\int_0^5 r_u du}$. Then finally G_1 is determined by taking the average of the discount factor multiplied with the indicator function:

$$\hat{G}_1(x) = \frac{1}{N} \sum_{all \text{ paths}} df(5) \cdot \mathbf{1}_{\{P(5,6) \leq x\}}$$

Its essentially the same for G_2 , except that we also multiply with the ZCB-price:

$$\hat{G}_1(x) = \frac{1}{N} \sum_{all \text{ paths}} df(5) \cdot P(5, 6) \cdot \mathbf{1}_{\{P(5,6) \leq x\}}$$

In conclusion, we obtain the following

Quantity	Analytical value	Monte Carlo value	Difference
G_1	0.403960	0.407012	-0.003052
G_2	0.369532	0.372383	-0.002850

It is clear from the Monte Carlo simulation that the implementation of the G-transform is well supported.

For completeness, the assignment doesn't specifically require $t = 0$, so the G_1, G_2 values have been calculated for $t = 0, \dots, 5$. by letting x be a function of t . This results in the following plot

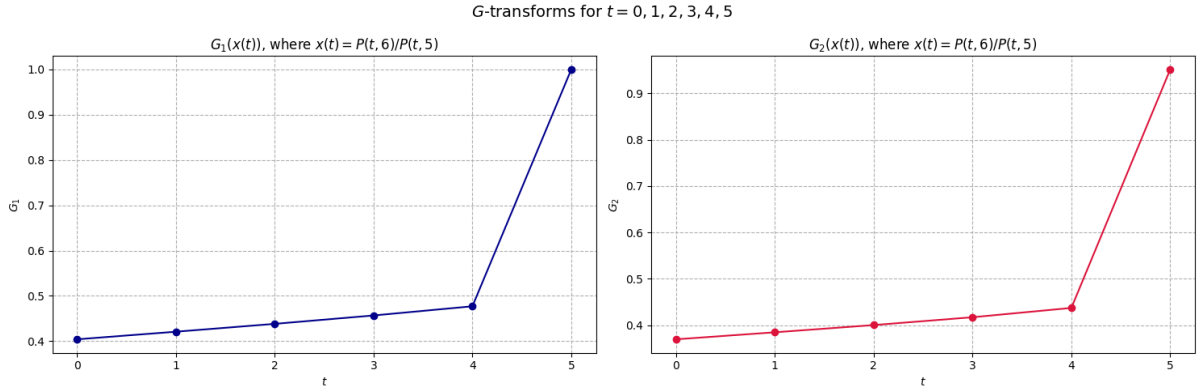


Figure 8: Values for G_1, G_2 for $t = 0, \dots, 5$

We omit verifying these values with Monte Carlo simulation.

7. For the ZCC firstly note that

$$\begin{aligned} [P(T, T + \Delta) - K]^+ &= P(T, T + \Delta) \mathbf{1}_{\{P(T, T + \Delta) > K\}} - K \mathbf{1}_{\{P(T, T + \Delta) > K\}} \\ &= P(T, T + \Delta) - P(T, T + \Delta) \mathbf{1}_{\{P(T, T + \Delta) < K\}} - K + K \mathbf{1}_{\{P(T, T + \Delta) < K\}} \end{aligned}$$

where we use that $1_A = 1 - 1_{A^c}$ for some set A . Therefore the price of ZCC is given by

$$\begin{aligned} \text{ZCC}(K, t, T, \Delta) &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} [P(T, T + \Delta) - K]^+ \right] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} (P(T, T + \Delta) - P(T, T + \Delta) \mathbf{1}_{\{P(T, T + \Delta) < K\}} - K + K \mathbf{1}_{\{P(T, T + \Delta) < K\}}) \right] \\ &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} P(T, T + \Delta) \right] - E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} P(T, T + \Delta) \mathbf{1}_{\{P(T, T + \Delta) < K\}} \right] \\ &\quad - K E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} \right] + K E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} \mathbf{1}_{\{P(T, T + \Delta) < K\}} \right] \\ &= P(t, T + \Delta) - G_2(K) - K P(t, T) + K G_1(K) \end{aligned}$$

For the put option (ZCP) we have by the same argument

$$\begin{aligned} ZCP(K, t, T, \Delta) &= E_t^{\mathbb{Q}} \left\{ e^{-\int_t^T r_u du} [K - P(T, T + \Delta)]^+ \right\} = E_t^{\mathbb{Q}} \left\{ e^{-\int_t^T r_u du} (K 1_{\{P(T, T + \Delta) \leq K\}} - P(T, T + \Delta)) 1_{\{P(T, T + \Delta) \leq K\}} \right\} \\ &= K E_t^{\mathbb{Q}} \left\{ e^{-\int_t^T r_u du} 1_{\{P(T, T + \Delta) \leq K\}} \right\} - E_t^{\mathbb{Q}} \left\{ e^{-\int_t^T r_u du} P(T, T + \Delta) 1_{\{P(T, T + \Delta) \leq K\}} \right\} \\ &= K G_1(K) - G_2(K) \end{aligned}$$

Therefore, the prices can easily be computed by calculating $G_1(K), G_2(K)$ with $K = 0.97$ as in 6. In addition, The Monte Carlo estimates are computed using the empirical discount factor For the call option, we estimate

$$\widehat{ZCC} = \frac{1}{N} \sum_{\text{all paths}} df(5) (P(5, 6) - K)^+,$$

and for the put option,

$$\widehat{ZCP} = \frac{1}{N} \sum_{\text{all paths}} df(5) (K - P(5, 6))^+.$$

The resulting prices are

Option	Analytical price	Monte Carlo price
ZCC	0.015915	0.015544
ZCP	0.022492	0.022602

So, Monte Carlo again verifies that the implementation of G-transforms used to derive the analytical prices of ZCC and ZCP are well supported.

Lastly we want to calculate $ZCC - ZCP$. Notice, that

$$\begin{aligned} ZCC(K) - ZCP(K) &= E_t^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} (P(T, T + \Delta) - K) \right] \\ &= P(t, T + \Delta) - KP(t, T) \end{aligned}$$

This is the put-call parity, and we should expect these to be the same. For $t = 0, T = 5, \Delta = 1$ and $K = 0.97$ we obtain

$$\begin{aligned} ZCC(0.97) - ZCP(0.97) &= -0.006577 \\ P(0, 6) - KP(0, 5) &= -0.006577 \end{aligned}$$

The result is negative. This happens since the strike $K = 0.97$ is above the forward price, since $P(0, 6) - KP(0, 5) < 0 \Leftrightarrow x(0) < K$. Therefore the put is more valuable than the call, which can also be seen in the option prices.

8. Let $t = 0, T = 5, \Delta = 1, n = 10$ and $d = 0.2$. This produces 11 put strikes k_i^p and 11 call strikes k_i^c , which are computed. Furthermore, we compute the put and call price for each put and price strike k_i^p, k_i^c respectively using

$$\begin{aligned} ZCP(k_i^p) &= k_i^p G_1(k_i^p) - G_2(k_i^p) \\ ZCC(k_i^c) &= P(0, 6) - k_i^c P(0, 5) - G_2(k_i^c) + k_i^c G_1(k_i^c) \end{aligned}$$

as in 2.7. The implied volatility of these options can now be determined by solving for the volatility parameter σ exactly when Black's price, from Black's formula, is equal to the model price. Here, Black's call formula is

$$C^{\text{Black}}(K, \sigma) = P(0, 5)(K^* \Phi(d_1) - K \Phi(d_2))$$

where

$$d_1 = \frac{\log(K^*/K) + 1/2 \cdot \sigma^2 T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T}$$

which are derived by roughly the same arguments as in 1.3. For the call strikes k_i^c the implied volatility $\sigma_{\text{imp}}(k_i^c)$ is therefore the solution to

$$C^{\text{Black}}(k_i^c, \sigma_{\text{imp}}) = ZCC(k_i^c)$$

For the put strikes k_i^p , we can convert the put price into the corresponding call price with the same strike by simple rearrangement. This results in the implied volatility $\sigma_{\text{imp}}(k_i^p)$ being the solution to

$$C^{\text{Black}}(k_i^p, \sigma_{\text{imp}}) = ZCP(k_i^c) + P(0, 6) - k_i^p P(0, 5) \quad (1)$$

The same thing is done, just with the added change of $\sigma_v = 2$ and $\rho = -0.25$. This results in the following plots

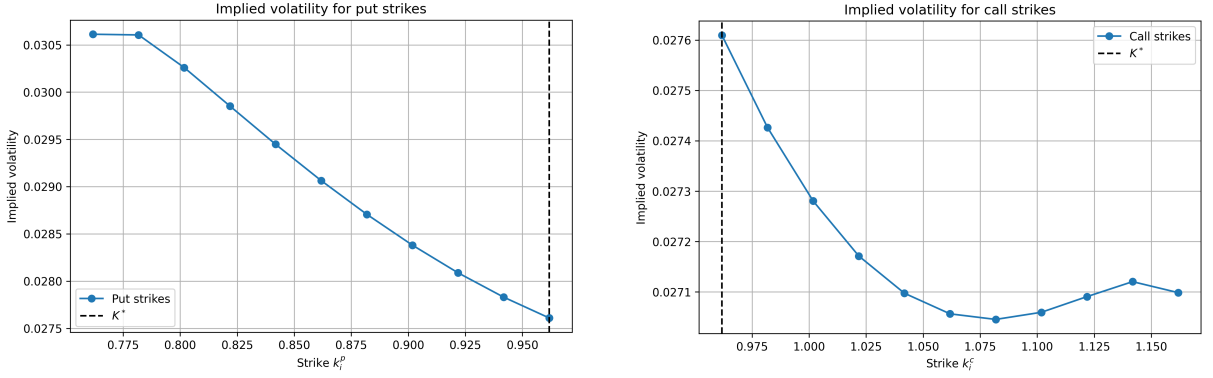


Figure 9: Implied volatility curves for the put strikes k_i^p and the call strikes k_i^c .

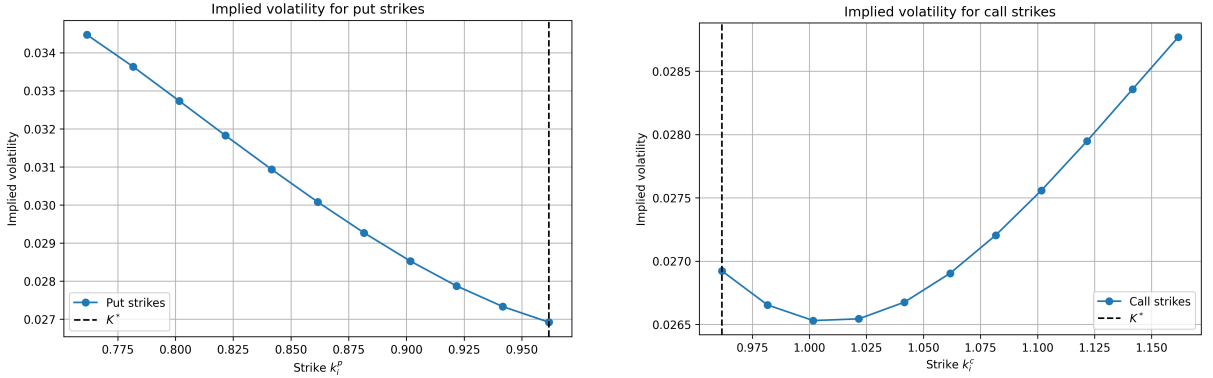


Figure 10: Implied volatility curves for the put strikes k_i^p and the call strikes k_i^c where $\sigma_v = 2$.

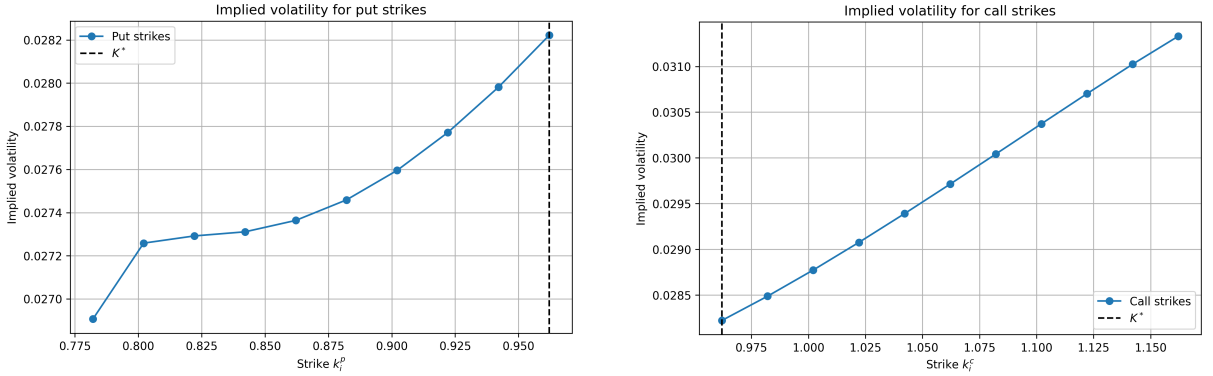


Figure 11: Implied volatility curves for the put strikes k_i^p and call strikes k_i^c when $\rho = -0.25$.

The base model produces an asymmetric volatility smile. Here the implied volatility for the put strikes decreases as the strike approaches K^* , while the implied volatility decreases a bit for the call strikes where it grows as the call strikes grow. Now letting $\sigma_v = 2$ the smile is much clearer especially for the call strikes. This also makes sense since if σ_v grows then the volatility factor is also more volatile, which creates more uncertainty in future bond prices. This implies larger volatilities away from K^* . When $\rho = -0.25$, the implied volatility for both the put and call prices are increasing in the strike. Changing the correlation therefore affects the skew of the smile.

3 A multifactor affine model

1. For the drift we need to find K and μ such that the drift matches the drifts of $dr(t)$ and $dv(t)$. Choosing

$$K = \begin{pmatrix} \kappa_r & 0 \\ 0 & \kappa_v \end{pmatrix}$$

and

$$\mu = \begin{pmatrix} \theta \\ 1 \end{pmatrix}$$

we immediately get

$$K(\mu - X(t))dt = \begin{pmatrix} \kappa_r & 0 \\ 0 & \kappa_v \end{pmatrix} \left\{ \begin{pmatrix} \theta \\ 1 \end{pmatrix} - \begin{pmatrix} r(t) \\ v(t) \end{pmatrix} \right\} dt = \begin{pmatrix} \kappa_r(\theta - r(t))dt \\ \kappa_v(1 - v(t))dt \end{pmatrix}$$

hence the drift matches. By the same procedure for the diffusion term, choosing

$$\Sigma = \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix}, \quad S(X(t)) = \begin{pmatrix} \sqrt{v(t)} & 0 \\ 0 & \sqrt{v(t)} \end{pmatrix}, \quad dW^{\mathbb{Q}}(t) = \begin{pmatrix} dW_1^{\mathbb{Q}}(t) \\ dW_2^{\mathbb{Q}}(t) \end{pmatrix}$$

we get

$$\begin{aligned} \Sigma S(X(t))dW^{\mathbb{Q}}(t) &= \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix} \begin{pmatrix} \sqrt{v(t)} & 0 \\ 0 & \sqrt{v(t)} \end{pmatrix} \begin{pmatrix} dW_1^{\mathbb{Q}}(t) \\ dW_2^{\mathbb{Q}}(t) \end{pmatrix} = \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix} \begin{pmatrix} \sqrt{v(t)}dW_1^{\mathbb{Q}}(t) \\ \sqrt{v(t)}dW_2^{\mathbb{Q}}(t) \end{pmatrix} \\ &= \begin{pmatrix} \sigma_r \sqrt{v(t)}dW_1^{\mathbb{Q}}(t) \\ \sigma_v \sqrt{v(t)}dW_2^{\mathbb{Q}}(t) \end{pmatrix}, \end{aligned}$$

which shows that the diffusion matches. Hence by choosing the drift and diffusion as shown above, the two-dimensional affine model can be rewritten into the form

$$dX(t) = K(\mu - X(t))dt + \Sigma S(X(t))dW^{\mathbb{Q}}(t)$$

as desired.

2. By Girsanov the dynamics can be changed using

$$dW^{\mathbb{Q}}(t) = dW^{\mathbb{P}}(t) - \Gamma_t dt.$$

Hence the dynamics under the physical measure become

$$\begin{aligned} dX(t) &= K(\mu - X(t))dt + \Sigma S(X(t))dW^{\mathbb{Q}}(t) = dX(t) = K(\mu - X(t))dt + \Sigma S(X(t)) (dW^{\mathbb{P}}(t) - \Gamma_t dt) \\ &= [K(\mu - X(t)) - \Sigma S(X(t))\Gamma_t] dt + \Sigma S(X(t))dW^{\mathbb{P}}(t) \end{aligned}$$

For the completely affine market price of risk the slides week 4 page 62 gives Γ_t as

$$\Gamma_t = S(X(t))\gamma$$

where $\gamma = (\gamma_1, \gamma_2)^T$ and $S(X(t))$ defined as in problem 1. In turn this leads to the dynamics given by

$$\begin{aligned} dX(t) &= \{K(\mu - X(t)) - \Sigma S(X(t))\Gamma_t\} dt + \Sigma S(X(t))dW^{\mathbb{P}}(t) \\ &= \{K(\mu - X(t)) - \Sigma S(X(t))S(X(t))\gamma\} dt + \Sigma S(X(t))dW^{\mathbb{P}}(t) \\ &= \left\{ \begin{pmatrix} \kappa_r(\theta - r(t)) \\ \kappa_v(1 - v(t)) \end{pmatrix} - \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix} \begin{pmatrix} \sqrt{v(t)} & 0 \\ 0 & \sqrt{v(t)} \end{pmatrix} \begin{pmatrix} \sqrt{v(t)} & 0 \\ 0 & \sqrt{v(t)} \end{pmatrix} \begin{pmatrix} \gamma_1 \\ \gamma_2 \end{pmatrix} \right\} dt + \begin{pmatrix} \sigma_r \sqrt{v(t)}dW_1^{\mathbb{P}}(t) \\ \sigma_v \sqrt{v(t)}dW_2^{\mathbb{P}}(t) \end{pmatrix} \\ &= \left\{ \begin{pmatrix} \kappa_r(\theta - r(t)) \\ \kappa_v(1 - v(t)) \end{pmatrix} - \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix} \begin{pmatrix} v(t)\gamma_1 \\ v(t)\gamma_2 \end{pmatrix} \right\} dt + \begin{pmatrix} \sigma_r \sqrt{v(t)}dW_1^{\mathbb{P}}(t) \\ \sigma_v \sqrt{v(t)}dW_2^{\mathbb{P}}(t) \end{pmatrix} \\ &= \begin{pmatrix} \{\kappa_r(\theta - r(t)) - \sigma_r v(t)\gamma_1\} dt \\ \{\kappa_v(1 - v(t)) - \sigma_v v(t)\gamma_2\} dt \end{pmatrix} + \begin{pmatrix} \sigma_r \sqrt{v(t)}dW_1^{\mathbb{P}}(t) \\ \sigma_v \sqrt{v(t)}dW_2^{\mathbb{P}}(t) \end{pmatrix} \end{aligned}$$

as desired. Under the essentially affine market price specification, the slides week 4 page 64 gives

$$\Gamma_t = S(X_t)\gamma + S^-(X_t)\Gamma_2 X_t$$

with $S^-(X_t)$ defined as

$$S^-(X_t) = \begin{pmatrix} \frac{1}{\sqrt{v}} & 0 \\ 0 & \frac{1}{\sqrt{v}} \end{pmatrix}$$

when $v(t) > 0$ and else just a 0-matrix. Furthermore

$$\Gamma_2 = \begin{pmatrix} \Gamma_{11} & \Gamma_{12} \\ \Gamma_{21} & \Gamma_{22} \end{pmatrix}$$

is some matrix with constant entries. For $v(t) > 0$ this leads to the calculations

$$\begin{aligned} dX(t) &= \{K(\mu - X(t)) - \Sigma S(X(t))\Gamma_t\} dt + \Sigma S(X(t))dW^\mathbb{P}(t) \\ &= \{K(\mu - X(t)) - \Sigma S(X(t)) (S(X_t)\gamma + S^-(X_t)\Gamma_2 X_t)\} dt + \Sigma S(X(t))dW^\mathbb{P}(t) \\ &= \{K(\mu - X(t)) - \Sigma S(X(t))S(X_t)\gamma - \Sigma S(X(t))S^-(X_t)\Gamma_2 X_t\} dt + \Sigma S(X(t))dW^\mathbb{P}(t) \\ &= \left\{ \begin{pmatrix} \kappa_r(\theta - r(t)) \\ \kappa_v(1 - v(t)) \end{pmatrix} - \begin{pmatrix} \sigma_r v(t)\gamma_1 \\ \sigma_v v(t)\gamma_2 \end{pmatrix} - \begin{pmatrix} \sigma_r \sqrt{v(t)} \\ \sigma_v \sqrt{v(t)} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{v}} & 0 \\ 0 & \frac{1}{\sqrt{v}} \end{pmatrix} \begin{pmatrix} \Gamma_{11} & \Gamma_{12} \\ \Gamma_{21} & \Gamma_{22} \end{pmatrix} \begin{pmatrix} r(t) \\ v(t) \end{pmatrix} \right\} dt + \begin{pmatrix} \sigma_r \sqrt{v(t)} dW_1^\mathbb{P}(t) \\ \sigma_v \sqrt{v(t)} dW_2^\mathbb{P}(t) \end{pmatrix} \\ &= \left\{ \begin{pmatrix} \kappa_r(\theta - r(t)) \\ \kappa_v(1 - v(t)) \end{pmatrix} - \begin{pmatrix} \sigma_r v(t)\gamma_1 \\ \sigma_v v(t)\gamma_2 \end{pmatrix} - \begin{pmatrix} \sigma_r \sqrt{v(t)} \\ \sigma_v \sqrt{v(t)} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{v}} & 0 \\ 0 & \frac{1}{\sqrt{v}} \end{pmatrix} \begin{pmatrix} r(t)\Gamma_{11} + v(t)\Gamma_{12} \\ r(t)\Gamma_{21} + v(t)\Gamma_{22} \end{pmatrix} \right\} dt + \begin{pmatrix} \sigma_r \sqrt{v(t)} dW_1^\mathbb{P}(t) \\ \sigma_v \sqrt{v(t)} dW_2^\mathbb{P}(t) \end{pmatrix} \\ &= \left\{ \begin{pmatrix} \kappa_r(\theta - r(t)) \\ \kappa_v(1 - v(t)) \end{pmatrix} - \begin{pmatrix} \sigma_r v(t)\gamma_1 \\ \sigma_v v(t)\gamma_2 \end{pmatrix} - \begin{pmatrix} \sigma_r \sqrt{v(t)} \\ \sigma_v \sqrt{v(t)} \end{pmatrix} \begin{pmatrix} \frac{r(t)\Gamma_{11} + v(t)\Gamma_{12}}{\sqrt{v}} \\ \frac{r(t)\Gamma_{21} + v(t)\Gamma_{22}}{\sqrt{v}} \end{pmatrix} \right\} dt + \begin{pmatrix} \sigma_r \sqrt{v(t)} dW_1^\mathbb{P}(t) \\ \sigma_v \sqrt{v(t)} dW_2^\mathbb{P}(t) \end{pmatrix} \\ &= \left\{ \begin{pmatrix} \kappa_r(\theta - r(t)) \\ \kappa_v(1 - v(t)) \end{pmatrix} - \begin{pmatrix} \sigma_r v(t)\gamma_1 \\ \sigma_v v(t)\gamma_2 \end{pmatrix} - \begin{pmatrix} \sigma_r r(t)\Gamma_{11} + \sigma_r v(t)\Gamma_{12} \\ \sigma_v r(t)\Gamma_{21} + \sigma_v v(t)\Gamma_{22} \end{pmatrix} \right\} dt + \begin{pmatrix} \sigma_r \sqrt{v(t)} dW_1^\mathbb{P}(t) \\ \sigma_v \sqrt{v(t)} dW_2^\mathbb{P}(t) \end{pmatrix} \\ &= \left(\begin{pmatrix} \kappa_r(\theta - r(t)) - \sigma_r[v(t)\gamma_1 - r(t)\Gamma_{11} + v(t)\Gamma_{12}] \\ \kappa_v(1 - v(t)) - \sigma_v[v(t)\gamma_2 - r(t)\Gamma_{21} + v(t)\Gamma_{22}] \end{pmatrix} dt \right) + \begin{pmatrix} \sigma_r \sqrt{v(t)} dW_1^\mathbb{P}(t) \\ \sigma_v \sqrt{v(t)} dW_2^\mathbb{P}(t) \end{pmatrix} \end{aligned}$$

as desired. Note that if $v(t) > 0$ does not hold calculations are the same as in the completely affine case since $S^-(X_t) = 0$. The key differences between these risk specifications is that in the completely affine case the market price of risk only depends on $v(t)$. On the other hand in the essentially affine model the market price of risk depends on X_t hence both $r(t)$ and $v(t)$ therefore as seen in the calculations above the drift-term becomes more flexible which ultimately as stated in duffee (2002) leads to better forecasts of the future treasury yield.

3. Firstly note we want to rewrite the process to the standard affine SDE form given by

$$dX(t) = B(D - X(t))dt + \Sigma S(X(t))dW^\mathbb{P}(t).$$

Hence we need to find matrices B and D such that the drift of the dynamics above matches the drift of the completely affine market price of risk specification, which means

$$\begin{pmatrix} \kappa_r(\theta - r(t)) - \sigma_r v(t)\gamma_1 \\ \kappa_v(1 - v(t)) - \sigma_v v(t)\gamma_2 \end{pmatrix} = B(D - X(t)) \iff \begin{pmatrix} \kappa_r\theta - \kappa_r r(t) - \sigma_r v(t)\gamma_1 \\ \kappa_v - \kappa_v v(t) - \sigma_v v(t)\gamma_2 \end{pmatrix} = BD - BX(t)$$

Rearrang the second row term in the matrix to $\kappa_v - (\kappa_v - \sigma_v \gamma_2)v(t)$ it is immediately clear that the B matrix should be

$$B = \begin{pmatrix} \kappa_r & \sigma_r \gamma_1 \\ 0 & \kappa_v + \sigma_v \gamma_2 \end{pmatrix}$$

such that

$$-BX(t) = - \begin{pmatrix} \kappa_r & \sigma_r \gamma_1 \\ 0 & \kappa_v + \sigma_v \gamma_2 \end{pmatrix} \begin{pmatrix} r(t) \\ v(t) \end{pmatrix} = \begin{pmatrix} -\kappa_r r(t) - \sigma_r v(t)\gamma_1 \\ -\kappa_v v(t) - \sigma_v v(t)\gamma_2 \end{pmatrix}.$$

Therefore by the equation above this means that

$$BD = \begin{pmatrix} \kappa_r & \sigma_r \gamma_1 \\ 0 & \kappa_v + \sigma_v \gamma_2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \kappa_r \theta \\ \kappa_v \end{pmatrix}.$$

Hence we need to solve the system of equations for x and y given by

$$\begin{aligned} x\kappa_r + y\sigma_r\gamma_1 &= \kappa_r\theta \\ y(\kappa_v + \sigma_v\gamma_2) &= \kappa_v \end{aligned}$$

which have the solutions

$$\begin{aligned} y &= \frac{\kappa_v}{\kappa_v + \sigma_v\gamma_2} \\ x &= \theta - \frac{\kappa_v\sigma_r\gamma_1}{\kappa_r(\kappa_v + \sigma_v\gamma_2)} \end{aligned}$$

and in turn the matrix D is therefore

$$D = \begin{pmatrix} \theta - \frac{\kappa_v\sigma_r\gamma_1}{\kappa_r(\kappa_v + \sigma_v\gamma_2)} \\ \frac{\kappa_v}{\kappa_v + \sigma_v\gamma_2} \end{pmatrix}.$$

By these choices we get the desired standard SDE form given by

$$dX(t) = B(D - X(t))dt + \Sigma S(X(t))dW^\mathbb{P}(t).$$

This allows the use of the multivariate version of the trick used to solve the Ornstein-Uhlenbeck SDE. By Itô's formula we have

$$\begin{aligned} d(e^{Bt}X(t)) &= e^{Bt}dX(t) + Be^{Bt}X(t)dt \\ &= e^{Bt}(B(D - X(t)))dt + \Sigma S(X(t))dW^\mathbb{P}(t) + Be^{Bt}X(t)dt \\ &= Be^{Bt}Ddt + \Sigma S(X(t))dW^\mathbb{P}(t) \end{aligned}$$

then simply integrating and diving by e^{Bt} on both sides yields the solution to the SDE given by

$$X(t) = e^{-Bt}X(0) + \int_0^t Be^{-B(t-u)}Ddu + \int_0^t e^{-B(t-u)}\Sigma S(X(u))dW^\mathbb{P}(u).$$

and calculating the integral

$$\int_0^t Be^{-B(t-u)}Ddu = \int_0^t e^{-B(t-u)}duBD = (I - e^{-Bt})D$$

then noting that the last term is a stochastic integral with conditional expectation zero and taking the conditional expectation yields

$$E\{X(t + \Delta)|\mathcal{F}_t\} = e^{-B\Delta}X(t) + \int_t^{t+\Delta} Be^{-B(t+\Delta-u)}du = e^{-B\Delta}X(t) + (I - e^{-B\Delta})D = AX(t) + b$$

with

$$A = e^{-B\Delta}, \quad b = (I - e^{-B\Delta})D$$

as desired. For the unconditional mean since B is an upper triangle matrix the eigenvalues of B are κ_r and $\kappa_v + \sigma_v\gamma_2$ and by table 2 these are positive. This means by section 5 in Christensen et al (2015) that $X(t)$ is stationary hence we have

$$\begin{aligned} E^\mathbb{P}\{X\} &= \lim_{\Delta \rightarrow \infty} E^\mathbb{P}\{X(t + \Delta)|\mathcal{F}_t\} = \lim_{\Delta \rightarrow \infty} e^{-B\Delta}X + (I - e^{-B\Delta})D = 0 + D = D = \begin{pmatrix} \theta - \frac{\kappa_v\sigma_r\gamma_1}{\kappa_r(\kappa_v + \sigma_v\gamma_2)} \\ \frac{\kappa_v}{\kappa_v + \sigma_v\gamma_2} \end{pmatrix} \\ &\approx \begin{pmatrix} -0.1474 \\ 0.9868 \end{pmatrix} \end{aligned}$$

since $e^{-B\Delta} \rightarrow 0$.

4. Following the formula for the conditional variance in section 3 in Christensen et al (2014), we have

$$V^{\mathbb{P}}[X(t + \Delta)|X_t] = \int_t^{t+\Delta} e^{-B(t+\Delta-s)} \Sigma S(E[X(s)|X(t)]) S(E[X(s)|X(t)])^T \Sigma^T e^{-B^T(t+\Delta-s)} ds$$

where B is the matrix found in problem 3. Therefore, by following the algorithm of section 3.1 in Christensen et al. (2014), we first find the eigenvalue decomposition of B . Since B is upper triangular its eigenvalues are its diagonal entries $\phi_1 = 0.5$ and $\phi_2 = 1.215$ and the matrix of eigenvectors is given by

$$Q = \begin{pmatrix} 1 & \frac{0.1}{1.216-0.5} \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0.139665 \\ 0 & 1 \end{pmatrix}$$

such that

$$B = Q \Phi Q^{-1} = \begin{pmatrix} 1 & 0.139665 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0.5 & 0 \\ 0 & 1.216 \end{pmatrix} \begin{pmatrix} 1 & 0.139665 \\ 0 & 1 \end{pmatrix}^{-1}$$

Then we proceed with the (iii) step. Since there are no statedependent terms we only calculate the $\varepsilon_{i,j}^1$. These are given by

$$\begin{aligned} \varepsilon_{1,1}^1\left(\frac{1}{52}\right) &= \frac{1 - e^{-2(0.5)\frac{1}{52}}}{2(0.5)} \approx 0.019047 \\ \varepsilon_{1,2}^1\left(\frac{1}{52}\right) &= \varepsilon_{2,1}^1\left(\frac{1}{52}\right) = \frac{1 - e^{-(0.5+1.216)\frac{1}{52}}}{0.5 + 1.216} \approx 0.018913 \\ \varepsilon_{2,2}^1\left(\frac{1}{52}\right) &= \frac{1 - e^{-2(1.216)\frac{1}{52}}}{2(1.216)} \approx 0.018786 \end{aligned}$$

Next we compute the (iv) step of the algorithm in 3.1, where we calculate

$$S(E^{\mathbb{P}}[X(s)|X(t)]) S(E^{\mathbb{P}}[X(s)|X(t)])^T$$

but

$$S(X(t)) S(X(t))^T = \begin{pmatrix} v(t) & 0 \\ 0 & v(t) \end{pmatrix}$$

which means

$$S(E^{\mathbb{P}}[X_s|X(t)]) S(E^{\mathbb{P}}[X_s|X(t)])^T = \begin{pmatrix} E^{\mathbb{P}}[v(s)|X(t)] & 0 \\ 0 & E^{\mathbb{P}}[v(s)|X(t)] \end{pmatrix}$$

Since we have that $X(t)$ is equal to its long-term mean which was calculated in 3. and is given by $X(t) = D$, we immediately get

$$E^{\mathbb{P}}[v(s)|X(t) = D] = D_{21} \approx 0.9868$$

this yields

$$S(E^{\mathbb{P}}[X_s|X(t)]) S(E^{\mathbb{P}}[X_s|X(t)])^T = \begin{pmatrix} E^{\mathbb{P}}[v(s)|X(t)] & 0 \\ 0 & E^{\mathbb{P}}[v(s)|X(t)] \end{pmatrix} \approx \begin{pmatrix} 0.9868 & 0 \\ 0 & 0.9868 \end{pmatrix}.$$

Next we calculate

$$\begin{aligned} \Sigma S(E^{\mathbb{P}}[X_s|X(t)]) S(E^{\mathbb{P}}[X_s|X(t)])^T \Sigma^T &= \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix} \begin{pmatrix} 0.9868 & 0 \\ 0 & 0.9868 \end{pmatrix} \begin{pmatrix} \sigma_r & 0 \\ 0 & \sigma_v \end{pmatrix}^T \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 0.08 \end{pmatrix} \begin{pmatrix} 0.9868 & 0 \\ 0 & 0.9868 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 0.08 \end{pmatrix} \\ &= \begin{pmatrix} 0.9868 & 0 \\ 0 & 0.00631552 \end{pmatrix} \end{aligned}$$

but since we have already calculated the $\bar{D} = S(E^{\mathbb{P}}[X_s|X(t)])S(E^{\mathbb{P}}[X_s|X(t)])^T$ as well as $\Sigma S(E^{\mathbb{P}}[X_s|X(t)])S(E^{\mathbb{P}}[X_s|X(t)])^T \Sigma^T$ we only need to calculate $\bar{S}\bar{D}\bar{S}^T$, where $\bar{S} = Q^{-1}\Sigma$. This is given by

$$\begin{aligned}\bar{S}\bar{D}\bar{S}^T &= Q^{-1} \begin{pmatrix} 0.9868 & 0 \\ 0 & 0.00631552 \end{pmatrix} (Q^{-1})^T = \begin{pmatrix} 1 & -0.139665 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0.9868 & 0 \\ 0 & 0.00631552 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ -0.139665 & 1 \end{pmatrix}^{-1} \\ &= \begin{pmatrix} 0.986923 & -0.000882 \\ -0.000882 & 0.00631552 \end{pmatrix}.\end{aligned}$$

Next we calculate the individual elements of the $\bar{V}_{i,j}(\frac{1}{52})$ which are simply given by

$$\begin{aligned}\bar{V}_{1,1}(\frac{1}{52}) &= 0.986923 \cdot \varepsilon_{1,1}^1(\frac{1}{52}) = 0.986923 \cdot 0.019047 = 0.018795 \\ \bar{V}_{2,1}(\frac{1}{52}) &= \bar{V}_{1,2}(\frac{1}{52}) = -0.000882 \cdot \varepsilon_{2,1}^1(\frac{1}{52}) = -0.000882 \cdot 0.018913 = -0.00001664 \\ \bar{V}_{2,2}(\frac{1}{52}) &= 0.00631552 \cdot \varepsilon_{2,2}^1(\frac{1}{52}) = 0.00631552 \cdot 0.018786 = 0.00011861\end{aligned}$$

finally this is converted to the conditional expectation using equation (3) in Christensen et al. (2014)

$$\begin{aligned}V^{\mathbb{P}}[X(t+\Delta)|X_t] &= Q\bar{V}Q^T = \begin{pmatrix} 1 & 0.139665 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0.018795 & -0.000882 \\ -0.000882 & 0.00011861 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0.139665 & 1 \end{pmatrix} \\ &\approx \begin{pmatrix} 0.0187964 & -0.000000114 \\ -0.000000114 & 0.000118661 \end{pmatrix}\end{aligned}$$

as desired. Since we argued in problem 3 that X_t is stationary we immediately get by section 4 that in order to calculate the unconditional expectation we simply let $\Delta \rightarrow \infty$ in the calculations of $\varepsilon_{i,j}^1(\Delta)$ and $\bar{V}_{ij}(\Delta)$. This yields

$$\begin{aligned}\varepsilon_{1,1}^1(\infty) &= \frac{1-0}{1} = 1 \\ \varepsilon_{1,2}^1(\infty) &= \varepsilon_{2,1}^1(\infty) = \frac{1-0}{0.5+1.216} \approx 0.582750 \\ \varepsilon_{2,2}^1(\infty) &= \frac{1-0}{2(1.216)} \approx 0.4111842\end{aligned}$$

and

$$\begin{aligned}\bar{V}_{1,1}(\infty) &= 0.986923 \cdot \varepsilon_{1,1}^1(\infty) = 0.986923 \cdot 1 = 0.986923 \\ \bar{V}_{2,1}(\infty) &= \bar{V}_{1,2}(\infty) = -0.000882 \cdot \varepsilon_{2,1}^1(\infty) = -0.000882 \cdot 0.582750 = -0.000514 \\ \bar{V}_{2,2}(\infty) &= 0.00631552 \cdot \varepsilon_{2,2}^1(\infty) = 0.00631552 \cdot 0.4111842 = 0.00259695\end{aligned}$$

and then simply converting again

$$V^{\mathbb{P}}[X(t)] = Q\bar{V}(\infty)Q^T = \begin{pmatrix} 1 & 0.139665 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0.986923 & -0.000514 \\ -0.000514 & 0.00259695 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0.139665 & 1 \end{pmatrix} \approx \begin{pmatrix} 0.986872 & -0.000151 \\ -0.000151 & 0.002597 \end{pmatrix}$$

as desired

5. The issue with this model with respect to the Kalman filter is the important assumption that requires the transition matrix given by

$$X(t+\Delta) = AX(t) + b + \omega_t$$

to have it's noise term ω_t be a zero-mean term. But since $v(t)$ is a CIR/square-root process (due to the $\sqrt{v}$ term in the diffusion), by the slide 75 week 4 this assumption does not hold for $X(t)$. Instead as done in problem 3. and 4. the conditional mean and variance can be computed by the formulas in Christensen et al (2014). Using the conditional mean and variance then by simply moment matching and using

$$X(t+\Delta)|X(t) \approx N(AX(t) + b, V^{\mathbb{P}}[X(t+\Delta)|X(t)])$$

such that

$$X(t + \Delta)|X(t) = AX(t) + b + \omega_t$$

and letting the noise term $\omega_t \approx N(0, V^{\mathbb{P}}[X(t + \Delta)|X(t)])$ and then simply using this when doing the Kalman filtering is a way of adapting the method as shown in the slides of week 4 page 75.

6. In order to implement the Kalman filter we use the slide 37 of week 4. Note firstly that the transition equation was derived preaviously and is given by

$$X_{n+1} = AX_n + b + \omega_n$$

where $A = e^{-B\Delta}$, $b = (I - e^{-B\Delta})D$ and $\omega_n \sim N(0, Q_n)$ where $Q_n = V^{\mathbb{P}}[X_{n+1}|X_n]$. For the mearsu-ment equation note firstly that the bond prices are given by

$$P(t, t + \tau) = \exp(\alpha(\tau) + \beta_r(\tau)r(t) + \beta_v(\tau)v(t))$$

while the yields is given by

$$y(t, \tau) = -\frac{1}{\tau} \log(P(t, t + \tau))$$

and then simply inserting yields

$$y(t, \tau) = -\frac{\alpha(\tau)}{\tau} - \frac{\beta_r(\tau)}{\tau}r(t) - \frac{\beta_v(\tau)}{\tau}v(t)$$

then note by slide 32 of week 4 that the measurement equation is given by

$$y_n = HX_n + d + \epsilon_n$$

where ϵ_n is a zero-mean noise term with covariance s_r^2 . This means that for the maturity times τ_i the entries H_i and d_i must be given by

$$d_i = \left(-\frac{\alpha(\tau_i)}{\tau_i} \right), H_i = \left(-\frac{\beta_r(\tau_i)}{\tau_i} \quad -\frac{\beta_v(\tau_i)}{\tau_i} \right)$$

where α , β_r and β_v by the exact argument as in 2.2 are the solutions to the Recatti equations given by

$$\begin{aligned} \beta_r'(\tau) &= -\kappa_r \beta_r(\tau) - 1, \quad \beta_r(0) = 0 \\ \beta_v'(\tau) &= -\kappa_v \beta_v(\tau) + \frac{1}{2} (\sigma_r^2 \beta_r(\tau)^2 + \sigma_v^2 \beta_v(\tau)^2), \quad \beta_v(0) = 0 \\ \alpha'(\tau) &= \kappa_r \theta \beta_r(\tau) + \kappa_v \beta_v(\tau) \quad \alpha(0) = 0. \end{aligned}$$

Hence all necessary values A, b, H, d and Q_n are found and the Kalman filter recursion found in slide 37 of week 4 can be performed. The update step is done by doing the following calculations

$$\begin{aligned} V_n &= y_n(Hm_{n|n-1} + d) \\ S_n &= HP_{n|n-1}H^T + R \\ K_n &= P_{n|n-1}H^TS_n^{-1} \\ m_{n|n} &= m_{n|n-1} + K_nV_n \\ P_{n|n} &= P_{n|n-1} - K_nS_nK_n^T \end{aligned}$$

with the initial condition that $m_{1|0} = E^P\{X(t)\}$ and $P_{1|0} = V^{\mathbb{P}}[X(t)]$. While the prediction is done by

$$\begin{aligned} m_{n+1|n} &= Am_{n|n} + b \\ P_{n+1|n} &= AP_{n|n}A^T + Q_n \end{aligned}$$

where $Q_n = V^{\mathbb{P}}[X_{n+1}|X_n]$, where we also note that going from n to $n + 1$ in our model corresponds to going t_n to $t_n + \Delta$, as this is the "step-size".

This is done in R combined with the L-BFGS-B-algorithm as the optimizer for the maximum likelihood estimate which leads to the maximum likelihood estimate of the paramaters as well as the loglikelihood given by

Parameter	Estimate
κ_r	0.874100
κ_v	0.402800
θ	0.107400
σ_r	0.221900
σ_v	1.769300
γ_1	0.708100
γ_2	0.758300
s_r	0.001448
Log-likelihood	17269.13

Table 1: Maximum likelihood estimates and loglikelihood from the Kalman filter.

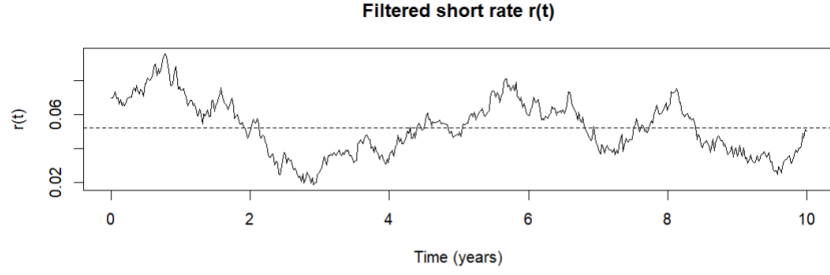


Figure 12: Short rate

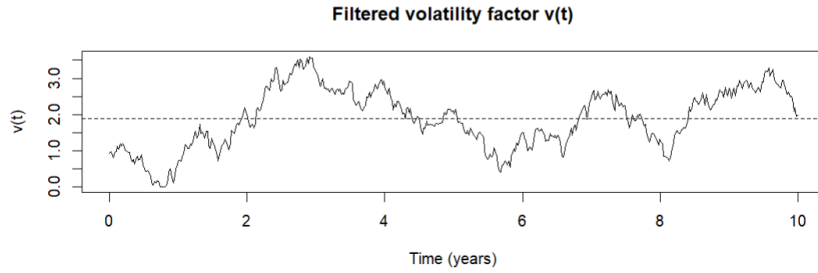


Figure 13: Volatility factor

The dashed lines shows the sample average of the two states. From the graph of the short rate factor it is clear that it does not fluctuate much in short periods of time but rather changes gradually while it remains high or low for extended periods of time. This is also consistent with the mean-reverting nature of the short rate. The volatility factor also varies a lot over time but compared to the short rate, the volatility has more pronounced swings with some elevations lasting more than 2 years. This indicates that the volatility factor deviates more from its mean. Therefore this could also indicate that the short rate factor captures general changes in the yield curve while the volatility factor captures more of the variation of the yield over time.

4 Python and R code

The code used throughout the assignment is a combination of Python (Question 2) and R (Question 3).
Python:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

#Drift parameters
kappa_r, theta_r = 0.5, 0.05
kappa_v, theta_v = 1.2, 1.0

#Volatility parameters
sigma_r = 0.08
sigma_v = 1.0
rho = -0.25

#alpha-beta parameters
alpha = 0.5
beta = 0.5

#initial state values
r_0 = theta_r
v_0 = theta_v

#from tau = 0 to tau = 30 years
tau_max = 30.0
tau_eval = np.linspace(0, tau_max, 301) #0.1 year steps

def riccati_system(tau, y):
    """
    """
    b_r, b_v, a_z = y

    #Common term
    vol_chunk = (sigma_r**2 * b_r**2) + (sigma_v**2 * b_v**2) + (2 * rho * sigma_r * sigma_v

    #ODEs
    db_r_dtau = -kappa_r * b_r - 1.0
    db_v_dtau = -kappa_v * b_v + 0.5 * beta * vol_chunk
    da_z_dtau = kappa_r * theta_r * b_r + kappa_v * theta_v * b_v + 0.5 * alpha * vol_chunk

    return [db_r_dtau, db_v_dtau, da_z_dtau]

#Initial boundary conditions at tau = 0: P(T,T) = 1 => alpha=0, beta_r=0, beta_v=0
init_conditions = [0.0, 0.0, 0.0]

#Use Runge-Kutta 45
sol = solve_ivp(
    fun=riccati_system,
    t_span=(0, tau_max),
    y0=init_conditions,
    t_eval=tau_eval,
    method='RK45'
)

#Extract solutions
beta_r_curve = sol.y[0]
beta_v_curve = sol.y[1]
alpha_curve = sol.y[2]
```

```

taus = sol.t

#P(0, 0+tau) = exp(alpha(tau) + beta_r(tau)*r_0 + beta_v(tau)*v_0 )
discount_curve = np.exp(alpha_curve + beta_r_curve * r_0 + beta_v_curve * v_0)

#Continuously compounded yield: y(0, 0+tau) = -log(P) / tau
yield_curve = np.zeros_like(discount_curve)
yield_curve[0] = r_0 #y(0) = r_0
yield_curve[1:] = -np.log(discount_curve[1:]) / taus[1:]

#Plots
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

#Plot 1: Riccati Beta Functions
ax1 = axs[0, 0]
ax2 = ax1.twinx()
p1 = ax1.plot(taus, beta_r_curve, label=r'$\beta_{z,r}(\tau)$ (Short rate)', color='crimson',
p2 = ax2.plot(taus, beta_v_curve, label=r'$\beta_{z,v}(\tau)$ (Volatility)', color='darkblue')
ax1.set_title("Riccati Coefficients: Beta Curves")
ax1.set_xlabel(r"Time-to-Maturity $\Delta$ (Years)")
ax1.set_ylabel("Short-rate Beta", color='crimson')
ax2.set_ylabel("Volatility Beta", color='darkblue')
ax1.grid(True, linestyle='--')
#Make y-axis ticks and spine colors match each curve
ax1.tick_params(axis='y', colors='crimson')
ax2.tick_params(axis='y', colors='darkblue')
ax1.spines['left'].set_color('crimson')
ax2.spines['right'].set_color('darkblue')
#Combine legends from both axes
lines = p1 + p2
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='center right')

#Plot 2: Riccati Alpha Function
axs[0, 1].plot(taus, alpha_curve, label=r'$\alpha_z(\tau)$ (Constant)', color='forestgreen',
axs[0, 1].set_title("Riccati Coefficients: Alpha Curve")
axs[0, 1].set_xlabel(r"Time-to-Maturity $\Delta$ (Years)")
axs[0, 1].set_ylabel("Alpha Value")
axs[0, 1].grid(True, linestyle='--')
axs[0, 1].legend()

#Plot 3: Discount Curve (ZCB Prices)
axs[1, 0].plot(taus, discount_curve, color='purple', lw=2.5)
axs[1, 0].set_title(r"Zero-Coupon Bond Price $P(0, \Delta)$")
axs[1, 0].set_xlabel(r"Maturity $\Delta$ (Years)")
axs[1, 0].set_ylabel("Bond Price")
axs[1, 0].set_ylim(0, 1.05)
axs[1, 0].grid(True, linestyle='--')

#Plot 4: Yield Curve
axs[1, 1].plot(taus, yield_curve * 100, color='darkorange', lw=2.5) # Convert to percentage
axs[1, 1].axhline(r_0 * 100, color='black', linestyle=':', label='Short Rate $r_0$')
axs[1, 1].set_title(r"Yield Curve $y(0, \Delta)$")
axs[1, 1].set_xlabel(r"Maturity $\Delta$ (Years)")
axs[1, 1].set_ylabel("Yield to Maturity (%)")
axs[1, 1].grid(True, linestyle='--')
axs[1, 1].legend()

plt.tight_layout()
plt.savefig("affine_diffusion_riccati_results.png")
plt.show()
%%

```

```

np.random.seed(42)

#Simulation Grid setup
T_max = 30
N_paths = 10000
N_steps = 1500
dt = T_max / N_steps
t_grid = np.linspace(0, T_max, N_steps + 1)

#Solve the ODEs on the same timeline as our simulation grid
sol = solve_ivp(riccati_system, (0, T_max), [0.0, 0.0, 0.0], t_eval=t_grid, method='RK45')
beta_r, beta_v, alpha_z = sol.y[0], sol.y[1], sol.y[2]

#Analytical prices: P(0, T)
P_analytical = np.exp(alpha_z + beta_r * r_0 + beta_v * v_0)

def MC_bond_price(T_max, N_paths, N_steps):
    r_paths = np.zeros((N_steps + 1, N_paths))
    v_paths = np.zeros((N_steps + 1, N_paths))
    log_D_paths = np.zeros((N_steps + 1, N_paths)) #Tracks the integrated running sum of -r_t

    r_paths[0, :] = r_0
    v_paths[0, :] = v_0

    for i in range(N_steps):
        #Current values at step i
        r_curr = r_paths[i, :]
        v_curr = v_paths[i, :]

        #Generate Correlated Brownian Shocks
        Z1 = np.random.normal(0, 1, N_paths)
        Z2 = np.random.normal(0, 1, N_paths)
        W_r = Z1
        W_v = rho * Z1 + np.sqrt(1 - rho**2) * Z2

        #Euler-Maruyama step transitions
        r_paths[i+1, :] = r_curr + kappa_r * (theta_r - r_curr) * dt + sigma_r * np.sqrt(alpha_z) * W_r
        v_paths[i+1, :] = v_curr + kappa_v * (theta_v - v_curr) * dt + sigma_v * np.sqrt(alpha_z) * W_v

        #Update cumulative discount path: log(D_t) = log(D_{t-1}) - r_t * dt
        log_D_paths[i+1, :] = log_D_paths[i, :] - r_curr * dt

    #Convert log discount paths back into absolute cash discount factors
    discount_paths = np.exp(log_D_paths)

    #For every maturity step T along the grid, average the sample paths horizontally
    p_monte_carlo = np.mean(discount_paths, axis=1)
    p_monte_carlo_err = np.std(discount_paths, axis=1) / np.sqrt(N_paths)

    return p_monte_carlo, p_monte_carlo_err, r_paths, v_paths, discount_paths

p_monte_carlo, p_monte_carlo_err, r_paths, v_paths, discount_paths = MC_bond_price(T_max, N_p

#Plots
plt.figure(figsize=(20, 6))
plt.plot(t_grid, P_analytical, label='Ricatti_Numerical_Solution', color='black', lw=3) #Plot
plt.errorbar(t_grid[::25], p_monte_carlo[::25], yerr=p_monte_carlo_err[::25], fmt='o', ecolor=
plt.title("Verification_of_Zero-Coupon_Bond_Prices_$P(0,T)$", fontsize=14)
plt.xlabel("Maturity_$T$(Years)", fontsize=12)
plt.ylabel("Bond_Price_$", fontsize=12)
plt.xlim(0, T_max)
plt.ylim(0, 1.05)
plt.grid(True, linestyle='--', alpha=0.6)

```

```

plt.legend(fontsize=12)
plt.savefig("monte_carlo_verification.png", dpi=300)
plt.show()

#5-year subset for plotting a sample of short-rate paths
idx_5 = np.argmin(np.abs(t_grid - 5.0))
t_grid_5 = t_grid[:idx_5+1]
r_paths_5 = r_paths[:idx_5+1, :]

#plot only sample r paths
plt.figure(figsize=(20, 6))
N_plot_paths = 20 #Number of random paths for plotting

#r_t paths plot
for j in range(N_plot_paths):
    plt.plot(t_grid_5, r_paths_5[:, j], lw=1.2, alpha=0.7)
plt.axhline(theta_r, color='black', linestyle='--', lw=2, label=r'Long-term Mean_\theta_r$')
plt.title("Sample Monte Carlo Fluctuations: Short Rate $r_t$", fontsize=13)
plt.ylabel("Interest Rate Value")
plt.xlabel("Time (Years)")
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.savefig("short_rate_sample_paths.png", dpi=300)
plt.show()

#evaluate forward bond prices for Delta = 1 and Delta = 10 years by doing standard forward di
T_eval = 5.0

idx = np.argmin(np.abs(t_grid - T_eval)) #Index where T = 5
idx_delta1 = np.argmin(np.abs(t_grid - (T_eval + 1))) #Index where T = 6
idx_delta10 = np.argmin(np.abs(t_grid - (T_eval + 10))) #index where T = 15

idx_d1 = np.argmin(np.abs(taus - 1.0))
idx_d10 = np.argmin(np.abs(taus - 10.0))

az_d1, bz_d1 = alpha_curve[idx_d1], np.array([beta_r_curve[idx_d1], beta_v_curve[idx_d1]])
az_d10, bz_d10 = alpha_curve[idx_d10], np.array([beta_r_curve[idx_d10], beta_v_curve[idx_d10]])

P_T_Tplus1 = np.exp(
    az_d1
    + bz_d1[0] * r_paths[idx, :]
    + bz_d1[1] * v_paths[idx, :]
)

P_T_Tplus10 = np.exp(
    az_d10
    + bz_d10[0] * r_paths[idx, :]
    + bz_d10[1] * v_paths[idx, :]
)

#Short rate at T = 5 across simulated paths
r_T = r_paths[idx, :]

#Empirical CDFs
vals1 = np.sort(P_T_Tplus1)
vals10 = np.sort(P_T_Tplus10)
vals_rT = np.sort(r_T)
n = P_T_Tplus1.size
ecdf = np.arange(1, n+1) / n

#Short rate plot
plt.figure(figsize=(7, 4))

```

```

plt.step(vals_rT, ecdf, where='post', label=r'$r_{T_{at}T=5}$', color='tab:green')
plt.xlabel(r'Short_Rate_$r_{T}$')
plt.ylabel('Cumulative_probability')
plt.title(r'$r_{T_{mid}X_0}$')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig('cdf_short_rate_T5.png')
plt.show()

fig, axs = plt.subplots(1, 3, figsize=(18, 5), sharey=False)

#First graph: P(T, T+1)
axs[0].step(vals1, ecdf, where='post', label='Delta_{1year}', color='tab:blue')
axs[0].set_xlabel('Forward_Zero-Coupon_Price_P(T,T+ )')
axs[0].set_ylabel('Cumulative_probability')
axs[0].set_title('P(T,T+1)')
axs[0].grid(True, linestyle='--', alpha=0.6)
axs[0].legend()

#Second graph: P(T, T+10)
axs[1].step(vals10, ecdf, where='post', label='Delta_{10years}', color='tab:orange')
axs[1].set_xlabel('Forward_Zero-Coupon_Price_P(T,T+ )')
axs[1].set_title('P(T,T+10)')
axs[1].grid(True, linestyle='--', alpha=0.6)
axs[1].legend()

#Third graph: both zero-coupon bond prices overlaid
axs[2].step(vals1, ecdf, where='post', label='Delta_{1year}', color='tab:blue')
axs[2].step(vals10, ecdf, where='post', label='Delta_{10years}', color='tab:orange')
axs[2].set_xlabel('Forward_Zero-Coupon_Price_P(T,T+ )')
axs[2].set_title('P(T,T+1) and P(T,T+10)')
axs[2].grid(True, linestyle='--', alpha=0.6)
axs[2].legend()

fig.suptitle('Empirical_CDFs_{at}T_{5years}', fontsize=14)
fig.tight_layout(rect=[0, 0, 1, 0.95])
plt.savefig('cdf_forward_bond_prices_and_short_rate.png')
plt.show()
#%%
from scipy.integrate import quad

def chf_riccati(tau, y):
    b_r, b_v, a_z = y

    #Common term
    vol_chunk = (sigma_r**2 * b_r**2) + (sigma_v**2 * b_v**2) + (2 * rho * sigma_r * sigma_v

    #ODEs
    db_r_dtau = -kappa_r * b_r
    db_v_dtau = -kappa_v * b_v + 0.5 * beta * vol_chunk
    da_z_dtau = kappa_r * theta_r * b_r + kappa_v * theta_v * b_v + 0.5 * alpha * vol_chunk

    return [db_r_dtau, db_v_dtau, da_z_dtau]

def compute_chf(k, T_eval, bz_delta):

    #Declare ricattii initial conditions
    initial_boundary = [1j * k * bz_delta[0], 1j * k * bz_delta[1], 0 + 0j]

    sol = solve_ivp(chf_riccati, [0, T_eval], initial_boundary, t_eval=[T_eval], method='RK45

    Br_final, Bv_final, A_final = sol.y[:, -1]

```

```

P_final = np.exp(A_final + Br_final * r_0 + Bv_final * v_0)

return P_final

def compute_psi(y, az_delta, bz_delta, T_eval):
    target = np.log(y) - az_delta

    def integrand(k):
        phi = compute_chf(k, T_eval, bz_delta)
        return np.imag(phi * np.exp(-1j * k * target)) / k

    integral, _ = quad(integrand, 1e-8, np.inf, limit=400, epsabs=1e-7, epsrel=1e-7)
    return 0.5 - (1 / np.pi) * integral

#alpha(Delta) and beta(Delta)
idx_d1 = np.argmin(np.abs(taus - 1.0)) #index where tau = 1
idx_d10 = np.argmin(np.abs(taus - 10.0)) #index where tau = 10

az_d1, bz_d1 = alpha_curve[idx_d1], np.array([beta_r_curve[idx_d1], beta_v_curve[idx_d1]])
az_d10, bz_d10 = alpha_curve[idx_d10], np.array([beta_r_curve[idx_d10], beta_v_curve[idx_d10]])

#Compute analytical cdf
y_vals = np.linspace(0.5, 1.5, 50)
psi1 = [compute_psi(y, az_d1, bz_d1, T_eval) for y in y_vals]
psi10 = [compute_psi(y, az_d10, bz_d10, T_eval) for y in y_vals]

#Plots
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

#Delta = 1
axs[0].step(vals1, ecdf, where='post', label='Empirical_MC_CDF', color='tab:blue', alpha=0.6,
axs[0].plot(y_vals, psi1, 'o--', label=r'Analytical_\psi_\Delta=1$Year', color='darkblue', mark
axs[0].set_title(r'ZCB_CDF:\Delta=1$Year')
axs[0].set_xlabel('Forward_Price_P(T,T+1)$')
axs[0].set_ylabel('Cumulative_Probability')
axs[0].set_xlim(0.25, 1.75)
axs[0].grid(True, linestyle='--')
axs[0].legend()

# Delta = 10
axs[1].step(vals10, ecdf, where='post', label='Empirical_MC_CDF', color='tab:orange', alpha=0.6,
axs[1].plot(y_vals, psi10, 'o--', label=r'Analytical_\psi_\Delta=10$Years', color='crimson', mark
axs[1].set_title(r'ZCB_CDF:\Delta=10$Years')
axs[1].set_xlabel('Forward_Price_P(T,T+10)$')
axs[1].set_xlim(0.25, 1.75)
axs[1].grid(True, linestyle='--')
axs[1].legend()

plt.suptitle(r'\psi(y;X_t,t,\Delta)_at_T=5$Years', fontsize=14)
plt.tight_layout()
plt.savefig('analytical_vs_empirical_cdf_verification.png')
plt.show()
%%
from scipy.integrate import quad

def compute_chf2(k, T_eval):
    k = np.asarray(k, dtype=complex)
    initial_boundary = [k[0], k[1], 0]

    sol = solve_ivp(
        riccati_system,

```

```

        [0, T_eval],
        initial_boundary,
        t_eval=[T_eval],
        method='RK45'
    )

    Br_final, Bv_final, A_final = sol.y[:, -1]
    P_final = np.exp(A_final + Br_final * r_0 + Bv_final * v_0)

    return P_final

def compute_G1(x, az_delta, bz_delta, T_eval, v_max=150, N=3000):
    target = np.log(x) - az_delta

    v_grid = np.linspace(1e-4, v_max, N)
    integrand_vals = []

    for v in v_grid:
        k = 1j * v * bz_delta
        phi = compute_chf2(k, T_eval)

        val = np.imag(phi * np.exp(-1j * v * target)) / v
        integrand_vals.append(val)

    integral = np.trapezoid(integrand_vals, v_grid)

    psi_0 = compute_chf2(np.array([0, 0]), T_eval)

    G1 = 0.5 * psi_0 - integral / np.pi

    return np.real(G1)

def compute_G2(x, az_delta, bz_delta, T_eval, v_max=150, N=3000):
    target = np.log(x) - az_delta

    v_grid = np.linspace(1e-4, v_max, N)
    integrand_vals = []

    for v in v_grid:
        k = (1 + 1j * v) * bz_delta
        phi = compute_chf2(k, T_eval)

        val = np.imag(phi * np.exp(-1j * v * target)) / v
        integrand_vals.append(val)

    integral = np.trapezoid(integrand_vals, v_grid)

    psi_beta = compute_chf2(bz_delta + 0j, T_eval)

    G2 = np.exp(az_delta) * (0.5 * psi_beta - integral / np.pi)

    return np.real(G2)

def get_zcb_coefficients(tau):
    alpha_tau = np.interp(tau, taus, alpha_curve)
    beta_r_tau = np.interp(tau, taus, beta_r_curve)
    beta_v_tau = np.interp(tau, taus, beta_v_curve)

    return alpha_tau, np.array([beta_r_tau, beta_v_tau])

```

```

def compute_chf2_state(k, T_eval, r_t, v_t):

    k = np.asarray(k, dtype=complex)
    initial_boundary = [k[0], k[1], 0]

    sol = solve_ivp(
        riccati_system,
        [0, T_eval],
        initial_boundary,
        t_eval=[T_eval],
        method='RK45'
    )

    Br_final, Bv_final, A_final = sol.y[:, -1]
    P_final = np.exp(A_final + Br_final * r_t + Bv_final * v_t)

    return P_final

def compute_G1_state(x, az_delta, bz_delta, T_eval, r_t, v_t, v_max=300, N=3000):
    target = np.log(x) - az_delta

    v_grid = np.linspace(1e-4, v_max, N)
    integrand_vals = []

    for v in v_grid:
        k = 1j * v * bz_delta
        phi = compute_chf2_state(k, T_eval, r_t, v_t)

        val = np.imag(phi * np.exp(-1j * v * target)) / v
        integrand_vals.append(val)

    integral = np.trapezoid(integrand_vals, v_grid)

    psi_0 = compute_chf2_state(np.array([0, 0]), T_eval, r_t, v_t)

    G1 = 0.5 * psi_0 - integral / np.pi

    return np.real(G1)

def compute_G2_state(x, az_delta, bz_delta, T_eval, r_t, v_t, v_max=300, N=3000):
    target = np.log(x) - az_delta

    v_grid = np.linspace(1e-4, v_max, N)
    integrand_vals = []

    for v in v_grid:
        k = (1 + 1j * v) * bz_delta
        phi = compute_chf2_state(k, T_eval, r_t, v_t)

        val = np.imag(phi * np.exp(-1j * v * target)) / v
        integrand_vals.append(val)

    integral = np.trapezoid(integrand_vals, v_grid)

    psi_beta = compute_chf2_state(bz_delta + 0j, T_eval, r_t, v_t)

    G2 = np.exp(az_delta) * (0.5 * psi_beta - integral / np.pi)

    return np.real(G2)

```

```

T_fixed = 5.0
Delta = 1.0

t_vals = np.array([0, 1, 2, 3, 4, 5], dtype=float)

G1_over_t = []
G2_over_t = []
x_over_t = []

# alpha_z(Delta), beta_z(Delta) for P(5,6)
az_delta, bz_delta = get_zcb_coefficients(Delta)

for t_val in t_vals:

    idx_t = np.argmin(np.abs(t_grid - t_val))

    # Use average simulated state at time t
    r_t = np.mean(r_paths[idx_t, :])
    v_t = np.mean(v_paths[idx_t, :])

    tau_T = T_fixed - t_val
    tau_TplusDelta = T_fixed + Delta - t_val

    # P(t,5)
    if tau_T == 0:
        P_t_T = 1.0
    else:
        az_T, bz_T = get_zcb_coefficients(tau_T)
        P_t_T = np.exp(
            az_T
            + bz_T[0] * r_t
            + bz_T[1] * v_t
        )

    # P(t,6)
    az_TplusDelta, bz_TplusDelta = get_zcb_coefficients(tau_TplusDelta)
    P_t_TplusDelta = np.exp(
        az_TplusDelta
        + bz_TplusDelta[0] * r_t
        + bz_TplusDelta[1] * v_t
    )

    # x(t) = P(t,6)/P(t,5)
    x_t = P_t_TplusDelta / P_t_T

    if tau_T == 0:
        # At t = T, no remaining discounting from t to T
        G1_t = 1.0
        G2_t = P_t_TplusDelta
    else:
        G1_t = compute_G1_state(
            x_t,
            az_delta,
            bz_delta,
            tau_T,
            r_t,
            v_t
        )

        G2_t = compute_G2_state(
            x_t,
            az_delta,
            bz_delta,

```

```

        tau_T,
        r_t,
        v_t
    )

    x_over_t.append(x_t)
    G1_over_t.append(G1_t)
    G2_over_t.append(G2_t)

x_over_t = np.array(x_over_t)
G1_over_t = np.array(G1_over_t)
G2_over_t = np.array(G2_over_t)

print("Number of t-values:", len(t_vals))
print("Number of G1-values:", len(G1_over_t))
print("Number of G2-values:", len(G2_over_t))

G_table = pd.DataFrame({
    "t": t_vals,
    "x(t)=P(t,6)/P(t,5)": x_over_t,
    "G1": G1_over_t,
    "G2": G2_over_t
})

print(G_table)

idx_t5 = np.argmin(np.abs(t_grid - 5.0))

# Empirical discount factor df(5)
D_T = discount_paths[idx_t5, :]

# Model bond price P(5,6) along each Monte Carlo path:
# P(5,6) = exp(alpha_z(1) + beta_z(1) * X_5)
P_T_Tplus1 = np.exp(
    az_delta
    + bz_delta[0] * r_paths[idx_t5, :]
    + bz_delta[1] * v_paths[idx_t5, :]
)

# The t = 0 value of x(t)
x_MC = x_over_t[0]

# Indicator condition {P(5,6) <= x}
indicator_MC = (P_T_Tplus1 <= x_MC)

# Monte Carlo estimates
G1_MC = np.mean(D_T * indicator_MC)
G2_MC = np.mean(D_T * P_T_Tplus1 * indicator_MC)

# Analytical values at t = 0 from the G-transform table
G1_analytical_t0 = G1_over_t[0]
G2_analytical_t0 = G2_over_t[0]

MC_verification_table = pd.DataFrame({
    "Quantity": ["G1", "G2"],
    "Analytical_value": [G1_analytical_t0, G2_analytical_t0],
    "Monte Carlo_value": [G1_MC, G2_MC],
    "Difference": [
        G1_analytical_t0 - G1_MC,
        G2_analytical_t0 - G2_MC
    ]
})

```

```

print("\nMonte Carlo verification at t=0:")
print(MC_verification_table)

fig, axs = plt.subplots(1, 2, figsize=(15, 5))

axs[0].plot(t_vals, G1_over_t, marker='o', linestyle='--', color='darkblue')
axs[0].set_title(r'$G_1(x(t))$, where $x(t)=P(t,6)/P(t,5)$')
axs[0].set_xlabel(r'$t$')
axs[0].set_ylabel(r'$G_1$')
axs[0].set_xticks(t_vals)
axs[0].grid(True, linestyle='--')

axs[1].plot(t_vals, G2_over_t, marker='o', linestyle='--', color='crimson')
axs[1].set_title(r'$G_2(x(t))$, where $x(t)=P(t,6)/P(t,5)$')
axs[1].set_xlabel(r'$t$')
axs[1].set_ylabel(r'$G_2$')
axs[1].set_xticks(t_vals)
axs[1].grid(True, linestyle='--')

plt.suptitle(r'$G$-transforms for $t=0,1,2,3,4,5$', fontsize=14)
plt.tight_layout()
plt.savefig(
    r"C:\Users\sogoj\Desktop\my_plot.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
###
# Question 7: ZCC and ZCP for K = 0.97

K = 0.97
T_eval = 5

# Since discount_curve is built directly from Riccati solutions, index over tau grid
idx_tau5 = np.argmin(np.abs(taus - 5)) # tau = 5
idx_tau6 = np.argmin(np.abs(taus - 6)) # tau = 6

P_0_5 = discount_curve[idx_tau5] # P(0,5)
P_0_6 = discount_curve[idx_tau6] # P(0,6)

# Riccati coefficients for Delta = 1
idx_tau1 = np.argmin(np.abs(taus - 1))
az_d1 = alpha_curve[idx_tau1]
bz_d1 = np.array([beta_r_curve[idx_tau1], beta_v_curve[idx_tau1]])

# Analytical G-transform values
g1_K = compute_G1(K, az_d1, bz_d1, T_eval)
g2_K = compute_G2(K, az_d1, bz_d1, T_eval)

# Analytical ZCC and ZCP prices
ZCC_analytical = P_0_6 - K * P_0_5 - g2_K + K * g1_K
ZCP_analytical = K * g1_K - g2_K

idx_t5 = np.argmin(np.abs(t_grid - 5))

D_T = discount_paths[idx_t5, :]

# P(5,6) = exp(alpha_z(1) + beta_z(1)*X_5)
P_T_Tplus1 = np.exp(
    az_d1
    + bz_d1[0] * r_paths[idx_t5, :]
    + bz_d1[1] * v_paths[idx_t5, :]

```

```

)

call_payoffs = np.maximum(P_T_Tplus1 - K, 0)
put_payoffs = np.maximum(K - P_T_Tplus1, 0)

ZCC_empirical = np.mean(D_T * call_payoffs)
ZCP_empirical = np.mean(D_T * put_payoffs)

# Put-call parity
parity_lhs = ZCC_analytical - ZCP_analytical
parity_rhs = P_0_6 - K * P_0_5

print(f"Analytical_ZCC_Price: {np.real(ZCC_analytical):.6f}")
print(f"Analytical_ZCP_Price: {np.real(ZCP_analytical):.6f}")
print(f"Empirical_ZCC_Price: {np.real(ZCC_empirical):.6f}")
print(f"Empirical_ZCP_Price: {np.real(ZCP_empirical):.6f}")
print(f"Put-Call_Parity_Check: LHS={np.real(parity_lhs):.6f}, RHS={np.real(parity_rhs):.6f}")
#%%
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import brentq

d = 0.2
n = 10
T_eval = 5.0

# Current state is t = 0
r_t = r_0
v_t = v_0

# Forward bond price K*
K_star = P_0_6 / P_0_5

i_grid = np.arange(0, n + 1)

kp_strikes = K_star - d * (1.0 - i_grid / n) # k_i^p
kc_strikes = K_star + d * (i_grid / n) # k_i^c

def black_call_zcb(K, sigma):
    if sigma <= 1e-12:
        return P_0_5 * max(K_star - K, 0.0)

    d1 = (np.log(K_star / K) + 0.5 * sigma ** 2 * T_eval) / (sigma * np.sqrt(T_eval))
    d2 = d1 - sigma * np.sqrt(T_eval)

    return P_0_5 * (K_star * norm.cdf(d1) - K * norm.cdf(d2))

def implied_vol_from_call(call_price, K):
    intrinsic = P_0_5 * max(K_star - K, 0.0)

    if call_price < intrinsic and abs(call_price - intrinsic) < 1e-8:
        call_price = intrinsic

    if call_price < intrinsic:
        print(f"No IV: K={K:.6f}, call_price={call_price:.10f}, intrinsic={intrinsic:.10f}")
        return np.nan

    def objective(sigma):
        return black_call_zcb(K, sigma) - call_price

```

```

low = 1e-10
high = 5.0

while objective(low) * objective(high) > 0 and high < 50:
    high *= 2

if objective(low) * objective(high) > 0:
    print(f"No root: K={K:.6f}, call_price={call_price:.10f}")
    return np.nan

return brentq(objective, low, high)

put_values = []

for i, K_i in zip(i_grid, kp_strikes):
    g1_K = compute_G1_state(
        K_i,
        az_d1,
        bz_d1,
        T_eval,
        r_t,
        v_t
    )

    g2_K = compute_G2_state(
        K_i,
        az_d1,
        bz_d1,
        T_eval,
        r_t,
        v_t
    )

    g1_K = np.real(g1_K)
    g2_K = np.real(g2_K)

    ZCP_i = K_i * g1_K - g2_K

    parity_call_i = ZCP_i + P_0_6 - K_i * P_0_5

    implied_vol_i = implied_vol_from_call(parity_call_i, K_i)

    black_reprice_i = (
        black_call_zcb(K_i, implied_vol_i)
        if not np.isnan(implied_vol_i)
        else np.nan
    )

    put_values.append({
        "i": i,
        "option": "put",
        "strike": K_i,
        "G1": g1_K,
        "G2": g2_K,
        "price": np.real(ZCP_i),
        "parity_call_price": np.real(parity_call_i),
        "implied_vol": implied_vol_i,
        "black_reprice_error": black_reprice_i - parity_call_i
    })

put_table = pd.DataFrame(put_values)

```

```

call_values = []

for i, K_i in zip(i_grid, kc_strikes):

    g1_K = compute_G1_state(
        K_i,
        az_d1,
        bz_d1,
        T_eval,
        r_t,
        v_t
    )

    g2_K = compute_G2_state(
        K_i,
        az_d1,
        bz_d1,
        T_eval,
        r_t,
        v_t
    )

    g1_K = np.real(g1_K)
    g2_K = np.real(g2_K)

    ZCC_i = P_0_6 - K_i * P_0_5 - g2_K + K_i * g1_K

    if ZCC_i < 0 and abs(ZCC_i) < 1e-8:
        ZCC_i = 0.0

    implied_vol_i = implied_vol_from_call(ZCC_i, K_i)

    black_reprice_i = (
        black_call_zcb(K_i, implied_vol_i)
        if not np.isnan(implied_vol_i)
        else np.nan
    )

    call_values.append({
        "i": i,
        "option": "call",
        "strike": K_i,
        "G1": g1_K,
        "G2": g2_K,
        "price": np.real(ZCC_i),
        "implied_vol": implied_vol_i,
        "black_reprice_error": black_reprice_i - ZCC_i
    })

call_table = pd.DataFrame(call_values)

option_values = pd.concat([put_table, call_table], ignore_index=True)

#Table
print("Put_table:")
print(put_table[[
    "i",
    "strike",
    "price",
    "parity_call_price",
    "implied_vol",
    "black_reprice_error"
]])

```

```

print("\nCall_table:")
print(call_table[[
    "i",
    "strike",
    "price",
    "implied_vol",
    "black_reprice_error"
]])

#Plots

plt.figure(figsize=(8, 5))

plt.plot(
    put_table["strike"],
    put_table["price"],
    marker="o",
    label="ZCP_values"
)

plt.plot(
    call_table["strike"],
    call_table["price"],
    marker="o",
    label="ZCC_values"
)

plt.axvline(K_star, linestyle="--", color="black", label=r"$K^*$")

plt.xlabel("Strike")
plt.ylabel("Option_value")
plt.title("Zero-coupon_bond_option_values")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    put_table["strike"],
    put_table["implied_vol"],
    marker="o",
    label="Put_strikes"
)

plt.axvline(K_star, linestyle="--", color="black", label=r"$K^*$")

plt.xlabel(r"Strike_$k_i^p$")
plt.ylabel("Implied_volatility")
plt.title("Implied_volatility_for_put_strikes")
plt.legend()
plt.grid(True)
plt.savefig(
    r"C:\Users\sogoj\Desktop\imp-volatility-put.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(

```

```

    call_table["strike"],
    call_table["implied_vol"],
    marker="o",
    label="Call strikes"
)

plt.axvline(K_star, linestyle="--", color="black", label=r"$K^*$")

plt.xlabel(r"Strike  $k_i^c$ ")
plt.ylabel("Implied volatility")
plt.title("Implied volatility for call strikes")
plt.legend()
plt.grid(True)
plt.savefig(
    r"C:\Users\sogoj\Desktop\imp-volatility-call.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

```

R-code for exercise 3.6:

```
library(expm)
library(readr)

data <- read_csv("data.csv", show_col_types = FALSE)

y <- as.matrix(data[, 2:8])

maturities <- c(1, 2, 3, 4, 5, 10, 20)
Delta <- 1 / 52

unpack <- function(z) {
  c(
    kappa_r = exp(z[1]),
    kappa_v = exp(z[2]),
    theta   = z[3],
    sigma_r = exp(z[4]),
    sigma_v = exp(z[5]),
    gamma_1 = z[6],
    gamma_2 = z[7],
    s_r     = exp(z[8])
  )
}

theta_to_z <- function(theta_vec) {
  c(
    log(theta_vec[1]), # kappa_r
    log(theta_vec[2]), # kappa_v
    theta_vec[3],      # theta
    log(theta_vec[4]), # sigma_r
    log(theta_vec[5]), # sigma_v
    theta_vec[6],      # gamma_1
    theta_vec[7],      # gamma_2
    log(theta_vec[8])  # s_r
  )
}

physical_matrices <- function(par, Delta = 1/52) {
  kr <- par["kappa_r"]
  kv <- par["kappa_v"]
  th <- par["theta"]
  sr <- par["sigma_r"]
  sv <- par["sigma_v"]
  g1 <- par["gamma_1"]
  g2 <- par["gamma_2"]

  KP <- matrix(
    c(
      kr, sr * g1,
      0, kv + sv * g2
    ),
    nrow = 2,
    byrow = TRUE
  )

  theta_v_P <- kv / (kv + sv * g2)
  theta_r_P <- th - (sr * g1 / kr) * theta_v_P

  thetaP <- c(theta_r_P, theta_v_P)

  A <- expm(-KP * Delta)
```

```

b <- as.numeric((diag(2) - A) %*% thetaP)

list(KP = KP, thetaP = thetaP, A = A, b = b)
}

bond_coefficients <- function(par, maturities, h = 0.005) {
  kr <- par["kappa_r"]
  kv <- par["kappa_v"]
  th <- par["theta"]
  sr <- par["sigma_r"]
  sv <- par["sigma_v"]

  max_tau <- max(maturities)
  grid <- seq(0, max_tau, by = h)

  alpha <- numeric(length(grid))
  beta_r <- numeric(length(grid))
  beta_v <- numeric(length(grid))

  f <- function(y_vec) {
    br <- y_vec[2]
    bv <- y_vec[3]

    da <- kr * th * br + kv * bv
    dbr <- -kr * br - 1
    dbv <- -kv * bv + 0.5 * (sr^2 * br^2 + sv^2 * bv^2)

    c(da, dbr, dbv)
  }

  y_vec <- c(0, 0, 0)

  for (i in 2:length(grid)) {
    k1 <- f(y_vec)
    k2 <- f(y_vec + 0.5 * h * k1)
    k3 <- f(y_vec + 0.5 * h * k2)
    k4 <- f(y_vec + h * k3)

    y_vec <- y_vec + h * (k1 + 2*k2 + 2*k3 + k4) / 6

    alpha[i] <- y_vec[1]
    beta_r[i] <- y_vec[2]
    beta_v[i] <- y_vec[3]
  }

  data.frame(
    tau = maturities,
    alpha = approx(grid, alpha, xout = maturities)$y,
    beta_r = approx(grid, beta_r, xout = maturities)$y,
    beta_v = approx(grid, beta_v, xout = maturities)$y
  )
}

measurement_matrices <- function(par, maturities) {
  bc <- bond_coefficients(par, maturities)

  tau <- as.numeric(maturities)
  alpha <- as.numeric(bc$alpha)
  beta_r <- as.numeric(bc$beta_r)

```

```

beta_v <- as.numeric(bc$beta_v)

H <- matrix(NA_real_, nrow = length(tau), ncol = 2)
H[, 1] <- -beta_r / tau
H[, 2] <- -beta_v / tau

d <- -alpha / tau

list(H = H, d = as.numeric(d))
}

conditional_covariance <- function(par, x, Delta = 1/52) {
  kr <- par["kappa_r"]
  kv <- par["kappa_v"]
  sr <- par["sigma_r"]
  sv <- par["sigma_v"]
  g1 <- par["gamma_1"]
  g2 <- par["gamma_2"]

  a <- kr
  cval <- sr * g1
  d <- kv + sv * g2

  if (!is.finite(a) || !is.finite(d) || a <= 0 || d <= 0) {
    return(NULL)
  }

  theta_v <- kv / d

  v0 <- max(as.numeric(x[2]), 1e-8)

  I_lambda <- function(lambda) {
    if (lambda <= 0 || !is.finite(lambda)) {
      return(NA_real_)
    }

    term1 <- theta_v * (1 - exp(-lambda * Delta)) / lambda

    denom <- lambda - d

    if (abs(denom) < 1e-10) {
      term2 <- (v0 - theta_v) * exp(-d * Delta) * Delta
    } else {
      term2 <- (v0 - theta_v) * exp(-d * Delta) *
        (1 - exp(-denom * Delta)) / denom
    }

    term1 + term2
  }

  I_2a <- I_lambda(2 * a)
  I_2d <- I_lambda(2 * d)
  I_ad <- I_lambda(a + d)

  if (any(!is.finite(c(I_2a, I_2d, I_ad)))) {
    return(NULL)
  }

  if (abs(d - a) < 1e-8) {

```

```

    return(NULL)
}

hcoef <- cval / (d - a)

Q22 <- sv^2 * I_2d
Q12 <- sv^2 * hcoef * (I_2d - I_ad)
Q11 <- sr^2 * I_2a +
    sv^2 * hcoef^2 * (I_2d - 2 * I_ad + I_2a)

Qn <- matrix(c(Q11, Q12, Q12, Q22), nrow = 2, byrow = TRUE)
Qn <- (Qn + t(Qn)) / 2

if (any(!is.finite(Qn))) {
    return(NULL)
}

eig <- eigen(Qn, symmetric = TRUE)
eig$values <- pmax(eig$values, 1e-12)
Qn <- eig$vectors %*% diag(eig$values) %*% t(eig$vectors)
Qn <- (Qn + t(Qn)) / 2

Qn
}

unconditional_covariance <- function(par) {
    mats <- physical_matrices(par)

    KP <- mats$KP
    thetaP <- mats$thetaP

    sr <- par["sigma_r"]
    sv <- par["sigma_v"]

    theta_v <- max(thetaP[2], 1e-8)

    M <- theta_v * diag(c(sr^2, sv^2))

    I2 <- diag(2)

    A_lyap <- kronecker(I2, KP) + kronecker(KP, I2)

    vecV <- tryCatch(
        solve(A_lyap, as.vector(M)),
        error = function(e) rep(NA_real_, 4)
    )

    V <- matrix(vecV, nrow = 2)
    V <- (V + t(V)) / 2

    if (any(!is.finite(V))) {
        return(NULL)
    }

    eig <- eigen(V, symmetric = TRUE)
    eig$values <- pmax(eig$values, 1e-12)
    V <- eig$vectors %*% diag(eig$values) %*% t(eig$vectors)
    V <- (V + t(V)) / 2

```

```

    v
  }

kalman_negloglik <- function(z, y, maturities, Delta = 1/52,
                             return_states = FALSE) {

  penalty <- 1e12

  par <- unpack(z)

  kr <- par["kappa_r"]
  kv <- par["kappa_v"]
  sr <- par["sigma_r"]
  sv <- par["sigma_v"]
  g2 <- par["gamma_2"]
  s_r <- par["s_r"]

  if (any(!is.finite(par))) return(penalty)
  if (kr <= 0 || kv <= 0 || sr <= 0 || sv <= 0 || s_r <= 0) return(penalty)
  if (kv + sv * g2 <= 0) return(penalty)

  matsP <- tryCatch(
    physical_matrices(par, Delta),
    error = function(e) NULL
  )
  if (is.null(matsP)) return(penalty)

  KP <- matsP$KP
  thetaP <- matsP$thetaP
  A <- matsP$A
  b <- matsP$b

  if (any(!is.finite(KP)) || any(!is.finite(thetaP))) return(penalty)
  if (any(Re(eigen(KP)$values) <= 0)) return(penalty)

  meas <- tryCatch(
    measurement_matrices(par, maturities),
    error = function(e) NULL
  )
  if (is.null(meas)) return(penalty)

  H <- meas$H
  d <- meas$d

  if (!all(dim(H) == c(length(maturities), 2))) return(penalty)
  if (length(d) != length(maturities)) return(penalty)
  if (any(!is.finite(H)) || any(!is.finite(d))) return(penalty)

  R <- diag(s_r^2, length(maturities))

  m <- as.numeric(thetaP)
  m[2] <- max(m[2], 1e-8)

  P <- unconditional_covariance(par)
  if (is.null(P)) return(penalty)

  loglik <- 0
  states <- matrix(NA_real_, nrow = nrow(y), ncol = 2)

```

```

for (n in 1:nrow(y)) {
  y_n <- as.numeric(y[n, ])

  y_hat <- as.numeric(d + H %%% m)
  nu <- y_n - y_hat

  Fmat <- H %%% P %%% t(H) + R
  Fmat <- (Fmat + t(Fmat)) / 2

  if (any(!is.finite(Fmat))) return(penalty)

  cholF <- tryCatch(chol(Fmat), error = function(e) NULL)
  if (is.null(cholF)) return(penalty)

  logdetF <- 2 * sum(log(diag(cholF)))

  ztmp <- forwardsolve(t(cholF), nu)
  quad <- sum(ztmp^2)

  k <- length(maturities)

  loglik <- loglik -
    0.5 * (k * log(2*pi) + logdetF + quad)

  K_gain <- P %%% t(H) %%% solve(Fmat)

  m_upd <- as.numeric(m + K_gain %%% nu)

  m_upd[2] <- max(m_upd[2], 1e-8)

  P_upd <- P - K_gain %%% Fmat %%% t(K_gain)
  P_upd <- (P_upd + t(P_upd)) / 2

  eigP <- eigen(P_upd, symmetric = TRUE)
  eigP$values <- pmax(eigP$values, 1e-12)
  P_upd <- eigP$vectors %%% diag(eigP$values) %%% t(eigP$vectors)
  P_upd <- (P_upd + t(P_upd)) / 2

  states[n, ] <- m_upd

  if (n < nrow(y)) {
    Qn <- conditional_covariance(par, m_upd, Delta)
    if (is.null(Qn)) return(penalty)

    m <- as.numeric(A %%% m_upd + b)
    m[2] <- max(m[2], 1e-8)

    P <- A %%% P_upd %%% t(A) + Qn
    P <- (P + t(P)) / 2

    eigP2 <- eigen(P, symmetric = TRUE)
    eigP2$values <- pmax(eigP2$values, 1e-12)
    P <- eigP2$vectors %%% diag(eigP2$values) %%% t(eigP2$vectors)
    P <- (P + t(P)) / 2
  }
}

if (return_states) {
  return(list(
    negloglik = -loglik,
    loglik = loglik,
    states = states,

```

```

        par = par
    ))
}

-loglik
}

theta_start <- c(
  0.8741,      # kappa_r
  0.4028,      # kappa_v
  0.1074,      # theta
  0.2219,      # sigma_r
  1.7693,      # sigma_v
  0.7081,      # gamma_1
  0.7583,      # gamma_2
  0.001448     # s_r
)

z_start <- theta_to_z(theta_start)

lower_theta <- c(
  0.01,        # kappa_r
  0.01,        # kappa_v
  -0.10,       # theta
  0.001,       # sigma_r
  0.001,       # sigma_v
  -5.00,       # gamma_1
  -5.00,       # gamma_2
  1e-6         # s_r
)

upper_theta <- c(
  5.00,        # kappa_r
  5.00,        # kappa_v
  0.30,        # theta
  3.00,        # sigma_r
  5.00,        # sigma_v
  5.00,        # gamma_1
  5.00,        # gamma_2
  0.05         # s_r
)

lower_z <- c(
  log(lower_theta[1]),
  log(lower_theta[2]),
  lower_theta[3],
  log(lower_theta[4]),
  log(lower_theta[5]),
  lower_theta[6],
  lower_theta[7],
  log(lower_theta[8])
)

upper_z <- c(
  log(upper_theta[1]),
  log(upper_theta[2]),
  upper_theta[3],
  log(upper_theta[4]),
  log(upper_theta[5]),
  upper_theta[6],

```

```

    upper_theta[7],
    log(upper_theta[8])
)

fit <- optim(
  par = z_start,
  fn = kalman_negloglik,
  y = y,
  maturities = maturities,
  method = "L-BFGS-B",
  lower = lower_z,
  upper = upper_z,
  control = list(
    maxit = 2000,
    factr = 1e7,
    pgtol = 1e-6,
    trace = 1
  )
)

res <- kalman_negloglik(
  fit$par,
  y = y,
  maturities = maturities,
  return_states = TRUE
)

theta_hat <- res$par
loglik_hat <- res$loglik
states_hat <- res$states

print(theta_hat)

print(loglik_hat)

time_years <- data[[1]]

par(mfrow = c(2, 1))

plot(
  time_years,
  states_hat[, 1],
  type = "l",
  main = "Filtered_short_rate_r(t)",
  xlab = "Time(years)",
  ylab = "r(t)"
)

abline(h = mean(states_hat[, 1]), lty = 2)

plot(
  time_years,
  states_hat[, 2],
  type = "l",
  main = "Filtered_volatility_factor_v(t)",

```

```
    xlab = "Time (years)",  
    ylab = "v(t)"  
)  
  
abline(h = mean(states_hat[, 2]), lty = 2)  
  
par(mfrow = c(1, 1))
```

References

- [1] Jean-François Le Gall (2016) *Brownian Motion, Martingales, and Stochastic Calculus*, Springer.
- [2] Duffie et. al (2000) *Transform Analysis and Asset Pricing For Affine Jump-diffusions*, Econometrica, Vol. 68, No. 6.